

Módulo 5. Actividades prácticas

Actividad práctica 1: clasificación de actividades educativas

Objetivo: Identificar oportunidades de aplicación de IA en el contexto educativo.

Alcance: Reflexionar sobre actividades comunes en clase y decidir cuáles podrían automatizarse.

Instrucciones:

En equipos de 5 participantes, discutan y enlisten 10 actividades frecuentes en su práctica docente.

Clasifíquenlas en: automatizable con IA / no automatizable.

Justifiquen cada clasificación en una tabla en Google Docs o Colab (Markdown).

Producto final: Tabla con 10 actividades clasificadas y justificadas.

# de Actividad	Nombre o descripción breve de la actividad	¿Automatizable con IA? (Sí, No, Parcialmente)	Breve justificación
1	Calificación de exámenes de opción múltiple	Sí	La IA puede corregir automáticamente mediante claves predefinidas, reduciendo tiempo y errores humanos
2	Explicación personalizada de conceptos complejos	Parcialmente	La IA puede ofrecer explicaciones básicas con chatbots o asistentes, pero no reemplaza del todo la interacción humana personalizada
3	Facilitación de debates en clase	No	La moderación de debates requiere habilidades sociales, gestión de emociones y adaptación que la IA aún no domina
4			
5			
6			
7			
8			
9			
10			

Actividad práctica 2: calculadora de calificaciones escolares con TensorFlow

Objetivo: Aplicar operaciones TensorFlow para simular un proceso educativo simple.

Alcance: Introducir tensores y operaciones básicas.

Instrucciones:

Abran un notebook en Google Colab.

Declaren tensores con calificaciones parciales.

Usen `tf.reduce_mean` para obtener promedios.

Simulen un sistema de evaluación con 3 actividades y nota final.

Producto final: Notebook funcional con cálculo de promedios usando TensorFlow.

Código:

```
# Paso 1: Importar TensorFlow
import tensorflow as tf

# Paso 2: Declarar tensores con calificaciones parciales (3 actividades
por estudiante)
# Simularemos 5 estudiantes
calificaciones = tf.constant([
    [85.0, 90.0, 78.0], # Estudiante 1
    [88.0, 92.0, 84.0], # Estudiante 2
    [75.0, 70.0, 80.0], # Estudiante 3
    [95.0, 94.0, 96.0], # Estudiante 4
    [60.0, 65.0, 58.0], # Estudiante 5
], dtype=tf.float32)
print("Calificaciones parciales (3 actividades por estudiante):")
print(calificaciones)

# Paso 3: Calcular el promedio simple por estudiante usando
tf.reduce_mean
promedios = tf.reduce_mean(calificaciones, axis=1)
print("\nPromedio por estudiante:")
print(promedios)

# Paso 4: Calcular nota final ponderada
# Supongamos que: A1 = 30%, A2 = 30%, A3 = 40%
ponderaciones = tf.constant([0.3, 0.3, 0.4], dtype=tf.float32)
notas_finales = tf.reduce_sum(calificaciones * ponderaciones, axis=1)
print("\nNota final por estudiante (ponderada):")
print(notas_finales)

# Paso 5: Evaluar quién aprueba (nota final >= 70)
aprobados = notas_finales >= 70
print("\n¿Aprobado? (True = aprobado, False = reprobado):")
print(aprobados)
```

```
# Paso 6: Mostrar resultados finales por estudiante
nombres = ["Estudiante 1", "Estudiante 2", "Estudiante 3", "Estudiante 4", "Estudiante 5"]
for i in range(len(nombres)):
    estado = "Aprobado" if aprobados[i].numpy() else "Reprobado"
    print(f"{nombres[i]}: Nota Final = {notas_finales[i].numpy():.2f} --> {estado}")
```

Explicación detallada del código:

Paso 1: Importar TensorFlow

```
import tensorflow as tf
```

Importa el módulo de TensorFlow con el alias `tf`. Esto es esencial para acceder a todas las funciones y estructuras que proporciona esta biblioteca para trabajar con tensores y operaciones matemáticas.

Paso 2: Declarar tensores con calificaciones parciales

```
calificaciones = tf.constant([
    [85.0, 90.0, 78.0], # Estudiante 1
    [88.0, 92.0, 84.0], # Estudiante 2
    [75.0, 70.0, 80.0], # Estudiante 3
    [95.0, 94.0, 96.0], # Estudiante 4
    [60.0, 65.0, 58.0], # Estudiante 5
], dtype=tf.float32)
```

¿Qué hace? Crea un tensor de 2 dimensiones (matriz) donde cada fila representa a un estudiante y cada columna representa una calificación de una actividad.

¿Por qué se usa `tf.constant`? Se utiliza para crear un tensor constante (sus valores no cambian durante la ejecución).

¿Qué es `dtype=tf.float32`? Define que los datos son de tipo número decimal (punto flotante de 32 bits), útil para operaciones matemáticas con precisión.

```
print("Calificaciones parciales (3 actividades por estudiante):")
print(calificaciones)
```

Lo que hace esta parte del código es que imprime el tensor para ver las calificaciones originales.

Paso 3: Calcular promedio por estudiante

```
promedios = tf.reduce_mean(calificaciones, axis=1)
```

Esta parte del código calcula el promedio de las tres actividades por estudiante, donde `axis=1` indica que se realiza la operación a lo largo de las columnas, es decir, se calcula la media de cada fila (estudiante).

```
print("\nPromedio por estudiante:")
print(promedios)
```

Esta parte del código lo que hace es mostrar los promedios simples para cada estudiante.

Paso 4: Calcular nota final ponderada

```
ponderaciones = tf.constant([0.3, 0.3, 0.4], dtype=tf.float32)
```

En esta parte, se define un tensor con los pesos (porcentajes) de cada actividad: la primera y segunda actividad valen 30% y la tercera actividad un 40%.

```
notas_finales = tf.reduce_sum(calificaciones * ponderaciones, axis=1)
```

En esta parte del código, se calcula la nota final ponderada para cada estudiante. Se realiza la multiplicación elemento a elemento entre cada fila de calificaciones y el vector de ponderaciones y finalmente, `reduce_sum(..., axis=1)` suma los valores por fila, obteniendo una nota final por estudiante.

```
print("\nNota final por estudiante (ponderada):")
print(notas_finales)
```

En esta parte se imprimen las notas finales ponderadas.

Paso 5: Evaluar quién aprueba

```
aprobados = notas_finales >= 70
```

En esta parte se genera un tensor booleano (`True` o `False`) que indica si cada estudiante aprueba ($\text{nota} \geq 70$).

```
print("\n¿Aprobado? (True = aprobado, False = reprobado):")
print(aprobados)
```

En esta parte se imprime un resumen indicando qué estudiantes aprobaron.

Paso 6: Mostrar resultados individuales

```
nombres = ["Estudiante 1", "Estudiante 2", "Estudiante 3", "Estudiante 4", "Estudiante 5"]
```

En esta parte, se crea una lista con los nombres de los estudiantes.

```
for i in range(len(nombres)):
    estado = "Aprobado" if aprobados[i].numpy() else "Reprobado"
    print(f"{nombres[i]}: Nota Final = {notas_finales[i].numpy():.2f} --> {estado}")
```

Esta última parte del código, lo que hace es recorrer cada estudiante para obtener la nota final como número real con `.numpy()`, determinar si aprueba o no y mostrar el nombre, la nota y el estado.

Actividad práctica 3: análisis de calificaciones de estudiantes

Objetivo: Visualizar datos educativos y comprender su distribución.

Alcance: Uso básico de pandas y matplotlib.

Instrucciones:

Crear los datos o cargar un archivo .csv desde Google Drive con calificaciones.

Usen pandas para leer y explorar los datos.

Grafiquen histogramas, cajas y diagramas de dispersión.

Producto final: Notebook con gráficas y análisis básico de un dataset educativo.

Código: (toma en cuenta la creación de los datos de forma manual usando pandas)

```
# Paso 1: Importar librerías necesarias
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

# Visualización en línea
%matplotlib inline

# Paso 2: Crear un dataset simulado directamente
data = {
    'nombre': ['Ana', 'Luis', 'Carlos', 'Laura', 'Pedro', 'Elena',
               'Mario', 'Sofía', 'Raúl', 'Diana'],
    'calificacion': [85, 78, 92, 68, 74, 88, 59, 91, 70, 83],
    'participacion': [90, 70, 85, 60, 75, 95, 40, 89, 65, 88]
}

df = pd.DataFrame(data)

# Paso 3: Exploración del dataset
print("Primeras filas del dataset:")
print(df.head())

print("\nResumen estadístico:")
print(df.describe())

# Paso 4: Histograma de calificaciones
plt.figure(figsize=(8, 5))
sns.histplot(df['calificacion'], bins=8, kde=True, color='skyblue')
plt.title('Distribución de Calificaciones')
plt.xlabel('Calificación')
plt.ylabel('Frecuencia')
plt.grid(True)
plt.show()

# Paso 5: Boxplot de calificaciones
plt.figure(figsize=(6, 4))
```

```

sns.boxplot(y=df['calificacion'], color='lightgreen')
plt.title('Boxplot de Calificaciones')
plt.ylabel('Calificación')
plt.grid(True)
plt.show()

# Paso 6: Diagrama de dispersión entre participación y calificación
plt.figure(figsize=(6, 4))
sns.scatterplot(x='participacion', y='calificacion', data=df,
color='coral')
plt.title('Dispersión: Participación vs Calificación')
plt.xlabel('Participación')
plt.ylabel('Calificación')
plt.grid(True)
plt.show()

# Paso 7: Convertir la columna de calificación a tensor de TensorFlow
calificaciones_tensor = tf.constant(df['calificacion'].values,
dtype=tf.float32)
print("\nTensores de calificaciones:")
print(calificaciones_tensor)

# Paso 8: Calcular promedio y desviación estándar con TensorFlow
promedio = tf.reduce_mean(calificaciones_tensor)
desviacion = tf.math.reduce_std(calificaciones_tensor)
print(f"\nPromedio con TensorFlow: {promedio.numpy():.2f}")
print(f"Desviación estándar con TensorFlow: {desviacion.numpy():.2f}")

```

Código: (toma en cuenta los datos de un archivo CSV cargado previamente en Google Drive)

```

# Paso 1: Instalar e importar librerías necesarias
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

# Activar visualización en línea para Colab
%matplotlib inline

# Paso 2: Conectar Google Drive para cargar el CSV
from google.colab import drive
drive.mount('/content/drive')

# Paso 3: Leer el archivo CSV desde Drive

```

```

# Asegúrate de que el archivo calificaciones.csv esté en tu Google Drive
ruta_csv = '/content/drive/MyDrive/calificaciones.csv' # Cambia esta
ruta si es necesario
df = pd.read_csv(ruta_csv)

# Paso 4: Exploración inicial del dataset
print("Información general del dataset:")
print(df.info())

print("\nPrimeras filas del dataset:")
print(df.head())

print("\nResumen estadístico de las calificaciones:")
print(df['calificacion'].describe())

# Paso 5: Histograma de calificaciones
plt.figure(figsize=(8, 5))
sns.histplot(df['calificacion'], bins=10, kde=True, color='skyblue')
plt.title('Distribución de Calificaciones')
plt.xlabel('Calificación')
plt.ylabel('Frecuencia')
plt.grid(True)
plt.show()

# Paso 6: Boxplot de calificaciones
plt.figure(figsize=(6, 4))
sns.boxplot(y=df['calificacion'], color='lightgreen')
plt.title('Boxplot de Calificaciones')
plt.ylabel('Calificación')
plt.grid(True)
plt.show()

# Paso 7: Diagrama de dispersión (si hay columna adicional como
participación)
if 'participacion' in df.columns:
    plt.figure(figsize=(6, 4))
    sns.scatterplot(x=df['participacion'], y=df['calificacion'],
color='coral')
    plt.title('Dispersión: Participación vs Calificación')
    plt.xlabel('Participación')
    plt.ylabel('Calificación')
    plt.grid(True)
    plt.show()
else:

```



```
print("No se encontró la columna 'participacion' para graficar  
dispersión.")
```

```
# Paso 8: Convertir calificaciones a tensor de TensorFlow  
calificaciones_tensor = tf.constant(df['calificacion'].values,  
dtype=tf.float32)  
print("\nTensores de calificaciones:\n", calificaciones_tensor)
```

Actividad práctica 4: Implementación de regresión lineal con TensorFlow

Objetivo: Aplicar un modelo de regresión para predecir resultados escolares.

Alcance: Introducción al aprendizaje supervisado.

Instrucciones:

Crear un dataset artificial: tareas entregadas vs. nota final.

Implementar regresión lineal con TensorFlow.

Visualizar la predicción y calcular el error.

Producto final: Notebook con modelo de regresión entrenado y validado.

Código:

```
# Paso 1: Importar librerías necesarias
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Paso 2: Crear un dataset artificial (tareas entregadas vs nota final)
# Supongamos 10 estudiantes
tareas_entregadas = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
dtype=float)
nota_final = np.array([50, 55, 60, 65, 68, 70, 75, 80, 85, 90],
dtype=float)

# Paso 3: Visualizar los datos
plt.scatter(tareas_entregadas, nota_final, color='blue')
plt.xlabel('Tareas entregadas')
plt.ylabel('Nota final')
plt.title('Relación entre tareas entregadas y nota final')
plt.grid(True)
plt.show()

# Paso 4: Normalizar los datos (opcional pero recomendable)
x = tareas_entregadas
y = nota_final

# Paso 5: Definir variables y modelo de regresión lineal
W = tf.Variable(0.0)
b = tf.Variable(0.0)

# Paso 6: Definir la función de predicción
def modelo_lineal(x):
    return W * x + b

# Paso 7: Definir la función de pérdida (MSE)
def loss(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
```

```

# Paso 8: Optimizador
optimizador = tf.optimizers.SGD(learning_rate=0.01)

# Paso 9: Entrenamiento del modelo
for epoch in range(500):
    with tf.GradientTape() as tape:
        pred = modelo_lineal(x)
        current_loss = loss(y, pred)
        grad = tape.gradient(current_loss, [W, b])
        optimizador.apply_gradients(zip(grad, [W, b]))

    if epoch % 50 == 0:
        print(f"Epoch {epoch}: pérdida = {current_loss.numpy():.4f}, W = {W.numpy():.4f}, b = {b.numpy():.4f}")

# Paso 10: Visualizar la línea de regresión
plt.scatter(x, y, label='Datos reales')
plt.plot(x, modelo_lineal(x), color='red', label='Regresión lineal')
plt.xlabel('Tareas entregadas')
plt.ylabel('Nota final')
plt.title('Modelo de regresión lineal')
plt.legend()
plt.grid(True)
plt.show()

# Paso 11: Predicción de una nueva nota con 7 tareas entregadas
nueva_tarea = 7.0
prediccion = modelo_lineal(nueva_tarea)
print(f"Predicción de nota para {nueva_tarea} tareas entregadas: {prediccion.numpy():.2f}")

# Paso 12: Calcular error cuadrático medio final
error_final = loss(y, modelo_lineal(x))
print(f"Error cuadrático medio final: {error_final.numpy():.2f}")

```

Explicación detallada del código:

Paso 1: Importar librerías necesarias

```

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

```

numpy (np): se importa porque se manejan arreglos numéricos en Python.

`tensorflow (tf)`: se debe importar para poder definir el modelo de aprendizaje automático y optimizarlo.

`matplotlib.pyplot (plt)`: se debe importar para poder visualizar los datos y la línea de regresión.

Paso 2: Crear dataset artificial

```
tareas_entregadas = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
dtype=float)
nota_final = np.array([50, 55, 60, 65, 68, 70, 75, 80, 85, 90],
dtype=float)
```

En esta parte, se generan dos arrays de 10 estudiantes con datos ficticios como ejemplo.

- o `tareas_entregadas`: número de tareas entregadas por cada estudiante.
- o `nota_final`: la nota correspondiente en cada actividad.

Paso 3: Visualizar los datos

```
plt.scatter(tareas_entregadas, nota_final, color='blue')
plt.xlabel('Tareas entregadas')
plt.ylabel('Nota final')
plt.title('Relación entre tareas entregadas y nota final')
plt.grid(True)
plt.show()
```

En esta parte, `plt.scatter` genera un gráfico de dispersión. Se observa visualmente una tendencia lineal entre ambas variables. Esto ayuda a motivar la construcción del modelo de regresión lineal.

Paso 4: Preparar datos para el modelo

```
x = tareas_entregadas
y = nota_final
```

En esta parte se renombran los datos para facilitar su uso posterior como `x` (entrada) y `y` (salida).

Paso 5: Inicializar los parámetros del modelo

```
W = tf.Variable(0.0)
b = tf.Variable(0.0)
```

En esta parte, se definen los parámetros del modelo y posteriormente estos se irán ajustando durante el entrenamiento.

- o `W`: pendiente de la recta.
- o `b`: intercepto o sesgo.

Paso 6: Función de predicción

```
def modelo_lineal(x):  
    return W * x + b
```

En esta parte se define el modelo de regresión lineal clásico: $y = Wx + b$. Esta función se usará para predecir notas en base a tareas entregadas.

Paso 7: Función de pérdida (MSE)

```
def loss(y_true, y_pred):  
    return tf.reduce_mean(tf.square(y_true - y_pred))
```

En esta parte, se define la función de pérdida que en este caso es MSE (Error cuadrático medio) la cual mide qué tan lejos están las predicciones (y_{pred}) de los valores reales (y_{true}). Se usa como criterio para ir ajustando los parámetros W y b durante el entrenamiento del modelo.

Paso 8: Definir optimizador

```
optimizador = tf.optimizers.SGD(learning_rate=0.01)
```

En este paso, se elige el optimizador SGD (descenso del gradiente estocástico), donde `learning_rate=0.01` indica qué tan grandes son los pasos para actualizar W y b .

Paso 9: Entrenamiento del modelo

```
for epoch in range(500):  
    with tf.GradientTape() as tape:  
        pred = modelo_lineal(x)  
        current_loss = loss(y, pred)  
        grad = tape.gradient(current_loss, [W, b])  
        optimizador.apply_gradients(zip(grad, [W, b]))
```

En esta parte lo que se hace es entrenar el modelo durante 500 iteraciones (epoch).

`GradientTape()` registra operaciones para calcular derivadas.

`grad` contiene los gradientes de la función de pérdida con respecto a W y b .

`apply_gradients` actualiza los valores de W y b para minimizar la pérdida.

```
if epoch % 50 == 0:  
    print(f"Epoch {epoch}: pérdida = {current_loss.numpy():.4f}, W =  
{W.numpy():.4f}, b = {b.numpy():.4f}")
```

En esta parte del código, cada 50 épocas se imprime la pérdida y los valores actuales de los parámetros. Esto permite monitorear el aprendizaje del modelo.

Paso 10: Visualizar la línea de regresión

```
plt.scatter(x, y, label='Datos reales')
```

```
plt.plot(x, modelo_lineal(x), color='red', label='Regresión lineal')
plt.xlabel('Tareas entregadas')
plt.ylabel('Nota final')
plt.title('Modelo de regresión lineal')
plt.legend()
plt.grid(True)
plt.show()
```

En esta parte se muestran los datos originales (puntos azules), la línea de regresión aprendida (línea roja) y visualmente se verifica si el modelo ajusta bien los datos.

Paso 11: Predicción con nuevo dato

```
nueva_tarea = 7.0
prediccion = modelo_lineal(nueva_tarea)
print(f"Predicción de nota para {nueva_tarea} tareas entregadas:
{prediccion.numpy():.2f}")
```

En esta parte lo que se hace es usar el modelo entrenado para predecir la nota de un estudiante que entregó 7 tareas y se imprime la nota estimada.

Paso 12: Calcular error final

```
error_final = loss(y, modelo_lineal(x))
print(f"Error cuadrático medio final: {error_final.numpy():.2f}")
```

En esta última parte del código, se calcula el MSE final para medir la precisión del modelo. Una pérdida baja indica un buen ajuste.

Como conclusión final, este código muestra a los profesores lo siguiente:

- ✓ La manera básica de implementar regresión lineal desde cero usando TensorFlow.
- ✓ El concepto de entrenamiento de un modelo supervisado.
- ✓ La forma cómo se optimiza una función de pérdida usando gradientes.
- ✓ El valor de visualizar los datos y el modelo.

Actividad práctica 5: “PrediNota” app para predecir calificaciones finales a partir de hábitos académicos

Objetivo: Diseñar una aplicación educativa interactiva que permita predecir la calificación final de un estudiante a partir de sus hábitos académicos usando un modelo de regresión con TensorFlow.

Alcance: Esta app puede ser utilizada por docentes o alumnos como herramienta de retroalimentación formativa, visualizando cómo variables como asistencia, tareas entregadas y participación afectan su rendimiento final.

Código:

```
# Paso 1: Importar bibliotecas necesarias
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider, VBox, Output

# Paso 2: Crear dataset simulado
np.random.seed(42)
data = pd.DataFrame({
    'asistencia': np.random.randint(50, 101, 100),
    'tareas': np.random.randint(50, 101, 100),
    'participacion': np.random.randint(50, 101, 100)
})
data['nota_final'] = 0.3 * data['asistencia'] + 0.4 * data['tareas'] +
0.3 * data['participacion']

# Paso 3: Normalización
X = data[['asistencia', 'tareas', 'participacion']].values / 100
y = data['nota_final'].values / 100

# Paso 4: Definir modelo
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(3,)),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, verbose=0)

# Paso 5: Interfaz interactiva
out = Output()

def predecir(asistencia, tareas, participacion):
    entrada = np.array([asistencia/100, tareas/100, participacion/100])
    pred = model.predict(entrada)[0][0] * 100
    with out:
        out.clear_output()
        print(f"Predicción de nota final: {pred:.2f} / 100")
```

```

slider1 = FloatSlider(min=50, max=100, step=1, value=75,
description='Asistencia')
slider2 = FloatSlider(min=50, max=100, step=1, value=75,
description='Tareas')
slider3 = FloatSlider(min=50, max=100, step=1, value=75,
description='Participación')

interact_ui = interact(predecir, asistencia=slider1, tareas=slider2,
participacion=slider3)
VBox([interact_ui.widget, out])

```

Explicación detallada:

Paso 1: Importar bibliotecas necesarias

```

import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider, VBox, Output

```

- **tensorflow:** es necesario para para construir y entrenar redes neuronales.
- **numpy:** es necesario para realizar operaciones numéricas eficientes.
- **pandas:** es necesario para manejar los datos como una tabla (DataFrame).
- **matplotlib.pyplot:** es necesario para graficar si se desea.
- **ipywidgets:** es necesario para crear controles interactivos (sliders) directamente en Colab o Jupyter.

Paso 2: Crear dataset simulado

```

np.random.seed(42)
data = pd.DataFrame({
    'asistencia': np.random.randint(50, 101, 100),
    'tareas': np.random.randint(50, 101, 100),
    'participacion': np.random.randint(50, 101, 100)
})
data['nota_final'] = 0.3 * data['asistencia'] + 0.4 * data['tareas'] +
0.3 * data['participacion'] + np.random.normal(0, 5, 100)

```

En esta parte, se generan 100 estudiantes simulados, con valores entre 50 y 100 para asistencia, tareas, participación. La nota final se calcula como una combinación ponderada: 30% asistencia + 40% tareas + 30% participación.

Paso 3: Normalización de datos

```

X = data[['asistencia', 'tareas', 'participacion']].values / 100
y = data['nota_final'].values / 100

```

En esta parte se normalizan los datos (escalar entre 0 y 1). Esto es necesario porque mejora el entrenamiento de redes neuronales. *x* será la matriz de entrada al modelo. *y* es la variable que queremos predecir (nota final), también normalizada.

Paso 4: Definir y entrenar el modelo

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(3,)),
    tf.keras.layers.Dense(1)
])
```

En esta parte del código, se crea un modelo de red neuronal con Keras, la API de alto nivel de TensorFlow, con la siguiente estructura:

- Capa oculta con 10 neuronas y activación ReLU, donde `input_shape=(3,)` quiere decir que cada estudiante tiene 3 variables de entrada.
- Capa de salida con 1 sola neurona (predicción continua).

```
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, verbose=0)
```

Con esta parte, lo que se hace es compilar el modelo con un optimizador `adam` (el cual es rápido y robusto) y una función de pérdida `MSE` (error cuadrático medio). Además, se entrena el modelo durante 100 épocas sobre los datos simulados.

Paso 5: Crear interfaz interactiva

```
out = Output()
```

Esta línea del código, permite imprimir mensajes dentro de la celda interactiva sin mezclarlo con otros prints.

```
def predecir(asistencia, tareas, participacion):
    entrada = np.array([[asistencia/100, tareas/100, participacion/100]])
    pred = model.predict(entrada)[0][0] * 100
    with out:
        out.clear_output()
        print(f"Predicción de nota final: {pred:.2f} / 100")
```

En esta parte, se define la función de predicción. Se reciben valores desde los sliders (`asistencia`, `tareas`, `participacion`), se normalizan los valores dividiéndolos entre 100, se hace la predicción con el modelo entrenado y por último se muestra la nota final estimada en el rango de 0 a 100.

```
slider1 = FloatSlider(min=50, max=100, step=1, value=75,
    description='Asistencia')
slider2 = FloatSlider(min=50, max=100, step=1, value=75,
    description='Tareas')
slider3 = FloatSlider(min=50, max=100, step=1, value=75,
    description='Participación')
```

En esta parte, se crean tres sliders (para la interfaz simple) con valores del 50 al 100 (igual que los datos simulados). Los valores por defecto están en 75.

```
interact_ui = interact(predecir, asistencia=slider1, tareas=slider2,
    participacion=slider3)
```

```
VBox([interact_ui.widget, out])
```

Por último, `interact()` conecta los sliders con la función `predecir` y `VBox()` organiza la interfaz visual para que los resultados aparezcan debajo de los sliders.

Actividad práctica 6: app para practicar derivadas con retroalimentación automática

Objetivo: Diseñar una interfaz educativa básica.

Alcance: Uso de ipywidgets y funciones condicionales.

Instrucciones:

Crear lista de preguntas con opciones de respuesta.

Mostrar cada pregunta con botones.

Evaluar respuestas e imprimir retroalimentación.

Producto final: App interactiva para ejercicios de derivadas.

Código:

```
# Paso 1: Instalar ipywidgets si no está disponible (solo necesario una vez)
!pip install ipywidgets --quiet
import ipywidgets as widgets
from IPython.display import display, clear_output
import tensorflow as tf

# Paso 2: Definir preguntas de opción múltiple sobre derivadas
preguntas = [
    {
        'pregunta': "¿Cuál es la derivada de  $f(x) = x^2$ ?",
        'opciones': [' $2x$ ', ' $x$ ', ' $x^3$ ', '1'],
        'respuesta_correcta': ' $2x$ '
    },
    {
        'pregunta': "¿Cuál es la derivada de  $f(x) = \sin(x)$ ?",
        'opciones': [' $\cos(x)$ ', ' $-\cos(x)$ ', ' $\sin(x)$ ', ' $-\sin(x)$ '],
        'respuesta_correcta': ' $\cos(x)$ '
    },
    {
        'pregunta': "¿Cuál es la derivada de  $f(x) = e^x$ ?",
        'opciones': [' $e^x$ ', ' $x \cdot e^{(x-1)}$ ', ' $\ln(x)$ ', ' $1/x$ '],
        'respuesta_correcta': ' $e^x$ '
    },
    {
        'pregunta': "¿Cuál es la derivada de  $f(x) = \ln(x)$ ?",
        'opciones': [' $1/x$ ', ' $\ln(x)$ ', ' $x$ ', ' $x \cdot \ln(x)$ '],
        'respuesta_correcta': ' $1/x$ '
    },
    {
        'pregunta': "¿Cuál es la derivada de  $f(x) = \cos(x)$ ?",
        'opciones': [' $-\sin(x)$ ', ' $\sin(x)$ ', ' $\cos(x)$ ', ' $-\cos(x)$ '],
        'respuesta_correcta': ' $-\sin(x)$ '
    }
]
```

```

    }
]

# Paso 3: Crear la interfaz con widgets
indice_actual = 0
output = widgets.Output()

def mostrar_pregunta(i):
    output.clear_output()
    with output:
        clear_output(wait=True)
        print(f"Pregunta {i + 1} de {len(preguntas)}")
        print(preguntas[i]['pregunta'])
        botones = []
        for opcion in preguntas[i]['opciones']:
            boton = widgets.Button(description=opcion)
            boton.on_click(lambda b, opcion=opcion:
evaluar_respuesta(opcion))
            botones.append(boton)
        box = widgets.VBox(botones)
        display(box)

def evaluar_respuesta(respuesta_usuario):
    global indice_actual
    output.clear_output()
    with output:
        correcta = preguntas[indice_actual]['respuesta_correcta']
        if respuesta_usuario == correcta:
            print("✓ ;Respuesta correcta!")
        else:
            print(f"✗ Incorrecto. La respuesta correcta es: {correcta}")
        indice_actual += 1
        if indice_actual < len(preguntas):
            boton_siguiente = widgets.Button(description="Siguiente")
            boton_siguiente.on_click(lambda b:
mostrar_pregunta(indice_actual))
            display(boton_siguiente)
        else:
            print("□ Has completado todas las preguntas.")

# Paso 4: Iniciar la app
display(output)
mostrar_pregunta(indice_actual)

```

Actividad práctica 7: “PlanTime” Estimador inteligente de tiempo necesario para estudiar

Objetivo: Crear una app educativa que estime el número de horas que un estudiante debe de estudiar para prepararse para un examen, tomando en cuenta la dificultad, el número de temas, su disponibilidad de tiempo y el nivel de dominio actual de los temas.

Alcance: Apoya la planificación del estudio personalizado, el cual es útil para las tutorías, autoevaluaciones y diseño de rutinas académicas.

Producto final: App interactiva implementada en Google Colab, que recibe parámetros del estudiante y devuelve una predicción razonable del tiempo necesario para estudiar.

Código:

```
# Paso 1: Importar bibliotecas necesarias
import tensorflow as tf
import numpy as np
import pandas as pd
from ipywidgets import interact, IntSlider, FloatSlider, VBox, Output
from IPython.display import display

# Paso 2: Dataset simulado
np.random.seed(123)
data = pd.DataFrame({
    'dificultad': np.random.randint(1, 6, 100),
    'temas': np.random.randint(5, 21, 100),
    'tiempo_disponible': np.random.randint(1, 6, 100),
    'dominio': np.random.rand(100)
})
data['horas_necesarias'] = (
    0.5 * data['dificultad'] +
    0.4 * data['temas'] -
    0.3 * data['tiempo_disponible'] -
    2.5 * data['dominio']
)
data['horas_necesarias'] = data['horas_necesarias'].apply(lambda x:
max(x, 1))

# Paso 3: Preparar datos
X = data[['dificultad', 'temas', 'tiempo_disponible', 'dominio']].values
y = data['horas_necesarias'].values

# Paso 4: Modelo de regresión
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation='relu', input_shape=(4,)),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, verbose=0)

# Paso 5: Interfaz interactiva
out = Output()
```

```
def estimar(dificultad, temas, tiempo_disponible, dominio):
    entrada = np.array([[dificultad, temas, tiempo_disponible, dominio]])
    pred = model.predict(entrada)[0][0]
    with out:
        out.clear_output()
        print(f"Horas recomendadas de estudio: {pred:.2f} h")

slider_dificultad = IntSlider(min=1, max=5, value=3,
description='Dificultad')
slider_temas = IntSlider(min=5, max=20, value=10, description='Temas')
slider_tiempo = IntSlider(min=1, max=5, value=3, description='Tiempo
Disp.')
```

```
slider_dominio = FloatSlider(min=0, max=1, step=0.01, value=0.5,
description='Dominio actual')

interact(estimar, dificultad=slider_dificultad, temas=slider_temas,
tiempo_disponible=slider_tiempo, dominio=slider_dominio)
display(out)
```

Explicación detallada:

Paso 1: Importar bibliotecas necesarias

```
import tensorflow as tf
import numpy as np
import pandas as pd
from ipywidgets import interact, IntSlider, FloatSlider, VBox, Output
from IPython.display import display
```

- **tensorflow:** es necesaria para construir y entrenar la red neuronal.
- **numpy:** es necesario para el manejo de arrays numéricos.
- **pandas:** es necesaria para crear y manipular el dataset simulado.
- **ipywidgets:** es necesario para crear controles interactivos (sliders) en la interfaz.
- **IPython.display.display:** es necesario para mostrar la salida interactiva en el entorno tipo Jupyter/Colab.

Paso 2: Crear dataset simulado

```
np.random.seed(123)
data = pd.DataFrame({
    'dificultad': np.random.randint(1, 6, 100),
    'temas': np.random.randint(5, 21, 100),
    'tiempo_disponible': np.random.randint(1, 6, 100),
    'dominio': np.random.rand(100)
})
```

En esta parte se generan datos simulados para 100 estudiantes con: **dificultad** (escala de 1 a 5), **temas** (entre 5 y 20 temas por unidad), **tiempo_disponible** (en una escala de 1 muy poco a 5 mucho) y **dominio** (un valor entre 0 y 1 que representa qué tanto conoce el contenido).

```
data['horas_necesarias'] = (
    0.5 * data['dificultad'] +
    0.4 * data['temas'] -
    0.3 * data['tiempo_disponible'] -
    2.5 * data['dominio']
)
```

Esta parte calcula la cantidad de horas necesarias con base en una fórmula simulada: Aumenta con la dificultad y los temas, disminuye con más tiempo disponible y mayor dominio.

```
data['horas_necesarias'] = data['horas_necesarias'].apply(lambda x:
max(x, 1))
```

Esta línea del código asegura que el valor mínimo de horas necesarias sea 1 hora.

Paso 3: Preparar datos

```
X = data[['dificultad', 'temas', 'tiempo_disponible', 'dominio']].values
y = data['horas_necesarias'].values
```

En esta parte se preparan los datos de x matriz con las 4 variables predictoras y los datos de y variable objetivo a predecir (horas necesarias de estudio).

Paso 4: Definir modelo de regresión

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation='relu', input_shape=(4,)),
    tf.keras.layers.Dense(1)
])
```

Esta parte del código crea una red neuronal con la siguiente estructura: una capa oculta de 8 neuronas con activación ReLU, una capa de salida con 1 neurona (predice horas) y entrada de 4 variables (una por cada factor del estudiante).

```
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, verbose=0)
```

Esta parte del código compila el modelo con un optimizador `adam`, y una función de pérdida `mse` (error cuadrático medio). Además, entrena el modelo durante 100 épocas.

Paso 5: Crear interfaz interactiva

```
out = Output()
```

Esta línea del código permite mostrar el resultado justo debajo de los sliders sin desordenar la celda.

```
def estimar(dificultad, temas, tiempo_disponible, dominio):
    entrada = np.array([[dificultad, temas, tiempo_disponible, dominio]])
```

```

pred = model.predict(entrada)[0][0]
with out:
    out.clear_output()
    print(f"Horas recomendadas de estudio: {pred:.2f} h")

```

En esta parte, se define el modelo de predicción: se reciben los valores de los controles deslizantes, se construye un arreglo de entrada (1 fila, 4 columnas), se llama al modelo para hacer la predicción y por último se muestra las horas estimadas de estudio con 2 decimales.

```

slider_dificultad = IntSlider(min=1, max=5, value=3,
description='Dificultad')
slider_temas = IntSlider(min=5, max=20, value=10, description='Temas')
slider_tiempo = IntSlider(min=1, max=5, value=3, description='Tiempo
Disp.')
slider_dominio = FloatSlider(min=0, max=1, step=0.01, value=0.5,
description='Dominio actual')

```

En esta parte se crean los Sliders que permiten ajustar valores como lo haría un estudiante o asesor: nivel de dificultad (1–5), número de temas (5–20), tiempo disponible (1–5), nivel de dominio actual (0 a 1 con pasos de 0.01).

```

interact(estimar, dificultad=slider_dificultad, temas=slider_temas,
tiempo_disponible=slider_tiempo, dominio=slider_dominio)
display(out)

```

Por último, en esta parte se conecta la interfaz. Se conectan los sliders con la función `estimar` y se muestra la salida del modelo con `display(out)`.

Actividad práctica 8: “Recomienda+” App recomendadora de recursos educativos personalizados

Objetivo: Diseñar una app interactiva que recomiende el tipo de recurso más adecuado (video, lectura o actividad práctica) en función del estilo de aprendizaje y motivación del estudiante.

Alcance: La aplicación puede ser usada como asistente virtual para orientar a los estudiantes sobre qué tipo de recursos deben utilizar al iniciar el estudio de un tema, según sus preferencias y nivel emocional.

Código:

```
# Paso 1: Importar librerías
import tensorflow as tf
import numpy as np
import pandas as pd
from ipywidgets import interact, Dropdown, FloatSlider, VBox, Output
from IPython.display import display

# Paso 2: Dataset simulado
np.random.seed(1)
# Estilos: visual=0, auditivo=1, kinestésico=2
# Motivación: 0 (baja) a 1 (alta)
data = pd.DataFrame({
    'estilo': np.random.choice([0, 1, 2], 100),
    'motivacion': np.random.rand(100)
})
data['recurso'] = (
    data['estilo'] +
    (data['motivacion'] > 0.6).astype(int)
) % 3 # 0=video, 1=lectura, 2=actividad

# Paso 3: Preparar datos para entrenamiento
X = data[['estilo', 'motivacion']].values
y = tf.keras.utils.to_categorical(data['recurso'], num_classes=3)

# Paso 4: Modelo de clasificación
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation='relu', input_shape=(2,)),
    tf.keras.layers.Dense(3, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=100, verbose=0)

# Paso 5: Interfaz interactiva
estilo_dict = {'Visual': 0, 'Auditivo': 1, 'Kinestésico': 2}
recursos = ['Video', 'Lectura', 'Actividad Práctica']

out = Output()

def recomendar(estilo, motivacion):
    entrada = np.array([[estilo_dict[estilo], motivacion]])
    pred = model.predict(entrada)[0]
    recurso = recursos[np.argmax(pred)]
    with out:
        out.clear_output()
```

```
print(f"Recurso recomendado: {recurso}")

select_estilo = Dropdown(options=['Visual', 'Auditivo', 'Kinestésico'],
description='Estilo')
slider_mot = FloatSlider(min=0, max=1, step=0.01, value=0.5,
description='Motivación')

interact(recomendar, estilo=select_estilo, motivacion=slider_mot)
display(out)
```

Actividad práctica 9: “LectoTest” App para el diagnóstico del nivel de comprensión lectora.

Objetivo: Diseñar una app educativa que clasifique automáticamente el nivel de comprensión lectora de un estudiante (básico, intermedio o avanzado) a partir de respuestas dadas a preguntas de opción múltiple.

Alcance: Esta app puede usarse para diagnósticos iniciales o como retroalimentación posterior a sesiones de lectura, adaptando actividades según el nivel detectado.

Código:

```
# Paso 1: Importar librerías
import tensorflow as tf
import numpy as np
import pandas as pd
from ipywidgets import interact_manual, Dropdown, Output, VBox

# Paso 2: Dataset simulado
np.random.seed(10)
data = pd.DataFrame({
    'pregunta1': np.random.choice([0, 1], 100),
    'pregunta2': np.random.choice([0, 1], 100),
    'pregunta3': np.random.choice([0, 1], 100),
    'pregunta4': np.random.choice([0, 1], 100),
    'pregunta5': np.random.choice([0, 1], 100),
})
data['total'] = data.sum(axis=1)
data['nivel'] = pd.cut(data['total'], bins=[-1, 2, 4, 5],
labels=['básico', 'intermedio', 'avanzado'])

X = data.drop(columns=['total', 'nivel']).values
y_map = {'básico': 0, 'intermedio': 1, 'avanzado': 2}
y = tf.keras.utils.to_categorical(data['nivel'].map(y_map))

# Paso 3: Modelo de clasificación
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation='relu', input_shape=(5,)),
    tf.keras.layers.Dense(3, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=100, verbose=0)

# Paso 4: Interfaz interactiva
out = Output()

p1 = Dropdown(options=[('Incorrecta', 0), ('Correcta', 1)],
description='P1')
p2 = Dropdown(options=[('Incorrecta', 0), ('Correcta', 1)],
description='P2')
p3 = Dropdown(options=[('Incorrecta', 0), ('Correcta', 1)],
description='P3')
p4 = Dropdown(options=[('Incorrecta', 0), ('Correcta', 1)],
description='P4')
p5 = Dropdown(options=[('Incorrecta', 0), ('Correcta', 1)],
description='P5')
```

```

niveles = ['básico', 'intermedio', 'avanzado']

@interact_manual(p1=p1, p2=p2, p3=p3, p4=p4, p5=p5)
def diagnosticar(p1, p2, p3, p4, p5):
    entrada = np.array([p1, p2, p3, p4, p5])
    pred = model.predict(entrada)[0]
    nivel = niveles[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Nivel de comprensión lectora estimado: {nivel}")

VBox([out])

```

Actividad práctica 10: “EmotionLearn” App para analizar los sentimientos en opiniones sobre clases

Objetivo: Diseñar una app educativa que detecte si el sentimiento de una opinión escrita por un estudiante es positiva, negativa o neutral, usando NLP y modelos de clasificación.

Alcance: Permite a los docentes analizar el tono emocional de los comentarios de sus estudiantes, con el fin de ajustar sus estrategias didácticas.

Producto final: App interactiva en Google Colab donde el usuario ingresa una opinión y obtiene el análisis de sentimiento.

Código:

```
# Paso 1: Importar librerías necesarias
import tensorflow as tf
import numpy as np
import pandas as pd
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from ipywidgets import Text, Output, VBox, Button
from IPython.display import display

# Paso 2: Dataset simulado
opiniones = [
    "Me encantó la clase, todo fue claro y bien explicado",
    "No entendí nada, fue una pérdida de tiempo",
    "Estuvo regular, algunas partes fueron interesantes",
    "Excelente docente, aprendí mucho",
    "Muy aburrido, no volvería a tomar esta materia",
    "No estuvo tan mal, aunque podría mejorar"
]
etiquetas = ['positivo', 'negativo', 'neutral', 'positivo', 'negativo', 'neutral']

# Paso 3: Preparar los datos
tokenizer = Tokenizer(num_words=1000, oov_token='<OOV>')
tokenizer.fit_on_texts(opiniones)
seqs = tokenizer.texts_to_sequences(opiniones)
padded = pad_sequences(seqs, padding='post')

label_map = {'positivo': 0, 'negativo': 1, 'neutral': 2}
y = tf.keras.utils.to_categorical([label_map[e] for e in etiquetas],
num_classes=3)

# Paso 4: Definir y entrenar el modelo
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(1000, 16, input_length=padded.shape[1]),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.fit(padded, y, epochs=100, verbose=0)

# Paso 5: Interfaz interactiva
entrada = Text(description='Opinión:')
out = Output()
```

```

boton = Button(description='Analizar sentimiento')

rev_map = {v: k for k, v in label_map.items()}

def analizar_sentimiento(b):
    texto = entrada.value
    seq = tokenizer.texts_to_sequences([texto])
    pad = pad_sequences(seq, maxlen=padded.shape[1], padding='post')
    pred = model.predict(pad)[0]
    resultado = rev_map[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Sentimiento detectado: {resultado}")

boton.on_click(analizar_sentimiento)
display(VBox([entrada, boton, out]))

```

Explicación detallada:

Paso 1: Importar librerías necesarias

```

import tensorflow as tf
import numpy as np
import pandas as pd
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from ipywidgets import Text, Output, VBox, Button
from IPython.display import display

```

- **tensorflow:** es necesaria para crear y entrenar el modelo de red neuronal.
- **numpy, pandas:** es necesaria para manejo de datos y arrays.
- **Tokenizer, pad_sequences:** son herramientas de Keras para preprocesar texto.
- **train_test_split:** Para dividir datos (no se usa aquí, pero puede ser útil).
- **ipywidgets y IPython.display:** es necesario para crear una interfaz interactiva que permita probar el modelo con opiniones nuevas.

Paso 2: Dataset simulado

```

opiniones = [
    "Me encantó la clase, todo fue claro y bien explicado",
    "No entendí nada, fue una pérdida de tiempo",
    "Estuvo regular, algunas partes fueron interesantes",
    "Excelente docente, aprendí mucho",
    "Muy aburrido, no volvería a tomar esta materia",
    "No estuvo tan mal, aunque podría mejorar"
]
etiquetas = ['positivo', 'negativo', 'neutral', 'positivo', 'negativo', 'neutral']

```

En esta parte se introduce un dataset simulado por medio de una lista de opiniones textuales de estudiantes. A cada opinión se le asigna una etiqueta de sentimiento: 'positivo', 'negativo', 'neutral'.

Paso 3: Preprocesamiento de texto

```
tokenizer = Tokenizer(num_words=1000, oov_token='<OOV>')
tokenizer.fit_on_texts(opiniones)
```

En esta parte del código se crea un tokenizador que convierte texto a secuencias numéricas.

- `num_words=1000`: considera solo las 1000 palabras más frecuentes.
- `oov_token='<OOV>'`: etiqueta especial para palabras desconocidas.

```
seqs = tokenizer.texts_to_sequences(opiniones)
padded = pad_sequences(seqs, padding='post')
```

En esta parte, lo que se hace es convertir cada texto en una secuencia de números. Luego, se rellenan (`padding='post'`) para que todas las secuencias tengan el mismo largo (importante para las redes neuronales).

```
label_map = {'positivo': 0, 'negativo': 1, 'neutral': 2}
y = tf.keras.utils.to_categorical([label_map[e] for e in etiquetas],
num_classes=3)
```

En esta parte, se transforma cada etiqueta textual en un número (0, 1, 2) y luego en un vector para entrenamiento.

Paso 4: Definir y entrenar el modelo

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(1000, 16, input_length=padded.shape[1]),
    tf.keras.layers.GlobalAveragePooling1D(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])
```

En esta parte, se crea la estructura del modelo de red neuronal: `Embedding` convierte cada palabra en un vector de 16 dimensiones, aprendidos durante el entrenamiento, `GlobalAveragePooling1D` reduce las dimensiones promediando los vectores de palabras, `Dense(16)` es una capa oculta con 16 neuronas y activación ReLU y `Dense(3, softmax)` es una capa de salida con 3 neuronas (una por clase: positivo, negativo, neutral), que da probabilidades.

```
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.fit(padded, y, epochs=100, verbose=0)
```

En esta parte se compila el modelo con `categorical_crossentropy` (ideal para clasificación multiclase) y `adam` como optimizador. Además, se entrena durante 100 épocas sin mostrar progreso (`verbose=0`).

Paso 5: Interfaz interactiva

```
entrada = Text(description='Opinión:')
out = Output()
boton = Button(description='Analizar sentimiento')
```

En esta parte se inicia la creación de la interfaz: `entrada` es la caja de texto donde el usuario escribe una nueva opinión, `boton` es el botón que activa el análisis y `out` es la zona de salida para mostrar el resultado.

```
rev_map = {v: k for k, v in label_map.items() }
```

Esta línea del código lo que hace es un mapeo reverso; es decir, convierte los números (0, 1, 2) de nuevo a etiquetas (positivo, negativo, neutral).

```
def analizar_sentimiento(b):
    texto = entrada.value
    seq = tokenizer.texts_to_sequences([texto])
    pad = pad_sequences(seq, maxlen=padded.shape[1], padding='post')
    pred = model.predict(pad)[0]
    resultado = rev_map[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Sentimiento detectado: {resultado}")
```

En esta parte se define la función que analizará el sentimiento. Cuando se hace clic toma el texto ingresado, lo convierte a secuencia y lo rellena, luego predice el sentimiento con el modelo entrenado y por último muestra el resultado (etiqueta con mayor probabilidad).

```
boton.on_click(analizar_sentimiento)
display(VBox([entrada, boton, out]))
```

Esta última parte del código lo que hace es mostrar la interfaz. Conecta el botón a la función `analizar_sentimiento`, organiza la interfaz verticalmente (`VBox`) y la muestra.

Actividad práctica 11: “OrientaTec” App para recomendar campos de estudio basado en intereses.

Objetivo: Diseñar una app que permita recomendar áreas académicas (ciencias, ingeniería, negocios, salud, diseño, etc) en función de los intereses personales del estudiante.

Alcance: Puede aplicarse en talleres de orientación vocacional en preparatorias o en programas de ingreso universitario o como parte de un módulo de orientación educativa.

Código:

```
# Paso 1: Importar librerías
import tensorflow as tf
import numpy as np
import pandas as pd
from ipywidgets import Checkbox, Button, Output, VBox
from IPython.display import display

# Paso 2: Crear dataset simulado
intereses = ['resolver problemas', 'leer y escribir', 'ayudar a otros',
             'diseñar cosas', 'liderar proyectos', 'hacer experimentos']
datos = []
etiquetas = []

for _ in range(300):
    perfil = np.random.choice([0, 1], size=6)
    datos.append(perfil)
    if perfil[0] == 1 and perfil[5] == 1:
        etiquetas.append('Ciencias')
    elif perfil[1] == 1 and perfil[3] == 1:
        etiquetas.append('Diseño')
    elif perfil[2] == 1:
        etiquetas.append('Salud')
    elif perfil[4] == 1:
        etiquetas.append('Negocios')
    else:
        etiquetas.append('General')

data = pd.DataFrame(datos, columns=intereses)
y_map = {'Ciencias': 0, 'Diseño': 1, 'Salud': 2, 'Negocios': 3,
         'General': 4}
y = tf.keras.utils.to_categorical([y_map[e] for e in etiquetas],
                                   num_classes=5)

# Paso 3: Entrenamiento del modelo
X = data.values
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(6,)),
    tf.keras.layers.Dense(5, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X, y, epochs=100, verbose=0)

# Paso 4: Interfaz interactiva
out = Output()
checkboxes = [Checkbox(description=i) for i in intereses]
boton = Button(description='Recomendar campo de estudio')
```

```

rev_map = {v: k for k, v in y_map.items()}

def recomendar(b):
    entrada = np.array([[int(cb.value) for cb in checkboxes]])
    pred = model.predict(entrada)[0]
    campo = rev_map[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Campo de estudio recomendado: {campo}")

boton.on_click(recomendar)
display(VBox(checkboxes + [boton, out]))

```

Explicación detallada:

Paso 1: Importar librerías

```

import tensorflow as tf
import numpy as np
import pandas as pd
from ipywidgets import Checkbox, Button, Output, VBox
from IPython.display import display

```

- **tensorflow:** es necesario para crear y entrenar la red neuronal.
- **numpy:** es necesario para generar datos aleatorios y manipular arrays.
- **pandas:** es necesario para crear un DataFrame que organiza los datos como tabla.
- **ipywidgets:** es necesario para crear una interfaz con checkboxes y botones.
- **display:** Para mostrar la interfaz en un entorno interactivo como Jupyter o Google Colab.

Paso 2: Crear dataset simulado

```

intereses = ['resolver problemas', 'leer y escribir', 'ayudar a otros',
'diseñar cosas', 'liderar proyectos', 'hacer experimentos']
datos = []
etiquetas = []

```

En esta parte, se crea un dataset simulado mediante una lista de intereses clave de un estudiante (simula respuestas tipo test vocacional).

- **datos:** es una lista para almacenar los vectores binarios (1 = le interesa, 0 = no le interesa).
- **etiquetas:** es una lista para la categoría de carrera a la que se asigna el perfil.

```

for _ in range(300):
    perfil = np.random.choice([0, 1], size=6)
    datos.append(perfil)
    if perfil[0] == 1 and perfil[5] == 1:
        etiquetas.append('Ciencias')
    elif perfil[1] == 1 and perfil[3] == 1:
        etiquetas.append('Diseño')
    elif perfil[2] == 1:
        etiquetas.append('Salud')
    elif perfil[4] == 1:
        etiquetas.append('Negocios')
    else:
        etiquetas.append('General')

```

En esta parte se hacen las simulaciones de los perfiles. Se crean 300 perfiles simulados. Cada perfil tiene 6 valores binarios (intereses). Luego, se asigna una etiqueta de carrera basada en reglas simples:

- Ciencias: si le gusta resolver problemas y hacer experimentos.
- Diseño: si le gusta leer/escribir y diseñar cosas.
- Salud: si quiere ayudar a otros.
- Negocios: si le gusta liderar proyectos.
- General: si no cumple ninguna de las reglas anteriores.

```

data = pd.DataFrame(datos, columns=intereses)
y_map = {'Ciencias': 0, 'Diseño': 1, 'Salud': 2, 'Negocios': 3,
         'General': 4}
y = tf.keras.utils.to_categorical([y_map[e] for e in etiquetas],
num_classes=5)

```

Esta parte del código lo que hace es convertir los datos a una tabla con columnas nombradas. Se convierte cada etiqueta (texto) en un número (0 a 4) y luego en un vector para clasificación multiclase.

Paso 3: Entrenamiento del modelo

```

X = data.values
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(6,)),
    tf.keras.layers.Dense(5, activation='softmax')
])

```

En esta parte del código se crea la estructura de la red neuronal, donde X matriz de entrada con los intereses codificados en binario. Se define un modelo secuencial con:

- Capa oculta con 10 neuronas ReLU.
- Capa de salida con 5 neuronas y activación `softmax` (una para cada campo de estudio).

```
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=100, verbose=0)
```

En esta parte se compila y entrena el modelo con una función de pérdida `categorical_crossentropy` (clasificación multiclase), un optimizador `adam` y 100 épocas de entrenamiento (silencioso con `verbose=0`).

Paso 4: Interfaz interactiva

```
out = Output()
checkboxes = [Checkbox(description=i) for i in intereses]
boton = Button(description='Recomendar campo de estudio')
```

En esta parte se empieza a crear la interfaz, donde: `out` es el área de salida para mostrar la recomendación. Se crean 6 checkboxes, uno para cada interés y un botón que el usuario presiona para obtener una recomendación.

```
rev_map = {v: k for k, v in y_map.items() }
```

Esta línea del código permite hacer un mapeo reverso de las etiquetas al transformar de vuelta el número a nombre de carrera.

```
def recomendar(b):
    entrada = np.array([[int(cb.value) for cb in checkboxes]])
    pred = model.predict(entrada)[0]
    campo = rev_map[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Campo de estudio recomendado: {campo}")
```

En esta parte del código se define la función de recomendación. Se convierte los checkboxes en una lista binaria (1 = marcado, 0 = no). Se hace la predicción con el modelo entrenado. Se selecciona el índice con mayor probabilidad y lo convierte al nombre del campo y por último se muestra el resultado en pantalla.

```
boton.on_click(recomendar)
display(VBox(checkboxes + [boton, out]))
```

Por último, esta parte permite hacer la conexión con la interfaz. Asocia el botón a la función `recomendar` y organiza todo en una caja vertical (`VBox`) para mostrarlo de manera ordenada.

Actividad práctica 12: “CienciaQuiz” Clasificador automático de respuestas de opción múltiple en Ciencias

Objetivo: Diseñar una app que clasifique respuestas en preguntas de opción múltiple de Ciencias como correctas o incorrectas y proporcione retroalimentación inmediata al estudiante.

Alcance: Puede usarse para prácticas automáticas, autoevaluaciones o integración en plataformas LMS para retroalimentación formativa.

Código:

```
# Paso 1: Importar librerías necesarias
import tensorflow as tf
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from ipywidgets import Dropdown, Button, Output, VBox
from IPython.display import display

# Paso 2: Dataset simulado
preguntas = [
    {"pregunta": "¿Cuál es la fórmula del agua?", "opciones": ["H2O",
"CO2", "O2"], "correcta": 0},
    {"pregunta": "¿Cuál es la unidad de fuerza?", "opciones": ["Joule",
"Newton", "Pascal"], "correcta": 1},
    {"pregunta": "¿Cuál es el órgano principal de la circulación?",
"opciones": ["Pulmón", "Hígado", "Corazón"], "correcta": 2},
]

# Crear dataset
X = []
y = []
for q in preguntas:
    for i, opc in enumerate(q['opciones']):
        entrada = [0]*3
        entrada[i] = 1
        X.append(entrada)
        y.append(1 if i == q['correcta'] else 0)

X = np.array(X)
y = to_categorical(y, 2)

# Paso 3: Entrenamiento del modelo
model = tf.keras.Sequential([
    tf.keras.layers.Dense(6, activation='relu', input_shape=(3,)),
    tf.keras.layers.Dense(2, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=200, verbose=0)

# Paso 4: Interfaz interactiva
out = Output()
def crear_quiz(pregunta):
    opciones = pregunta["opciones"]
    dropdown = Dropdown(options=opciones,
description=pregunta["pregunta"][:20]+":")
    boton = Button(description="Verificar")

    def verificar(b):
```

```

idx = opciones.index(dropdown.value)
entrada = [0]*3
entrada[idx] = 1
pred = model.predict(np.array([entrada]))
resultado = "Correcto" if np.argmax(pred) == 1 else "Incorrecto"
with out:
    out.clear_output()
    print(f"Respuesta: {resultado}")

boton.on_click(verificar)
display(VBox([dropdown, boton, out]))

crear_quiz(preguntas[0])

```

Actividad práctica 13: “LearnStyle” Detección de estilo de aprendizaje basado en respuestas a cuestionarios

Objetivo: Desarrollar una app que prediga el estilo de aprendizaje dominante (visual, auditivo, kinestésico o mixto) a partir de las respuestas de un estudiante a un breve cuestionario.

Alcance: Es útil para orientar las estrategias de enseñanza del docente al inicio de un curso o como parte de un programa de tutoría académica.

Producto final: Interfaz interactiva de selección de opciones tipo checkbox que clasifica automáticamente el estilo de aprendizaje predominante.

Código:

```
# Paso 1: Importar bibliotecas
import tensorflow as tf
import numpy as np
import pandas as pd
from tensorflow.keras.utils import to_categorical
from ipywidgets import Checkbox, Button, VBox, Output
from IPython.display import display

# Paso 2: Crear dataset simulado
preguntas = ['Prefiero diagramas', 'Escucho podcasts', 'Aprendo
moviéndome',
              'Hago mapas mentales', 'Recuerdo lo que escucho', 'Necesito
tocar']
datos = []
etiquetas = []

for _ in range(300):
    perfil = np.random.choice([0, 1], size=6)
    datos.append(perfil)
    if perfil[0] + perfil[3] > 1:
        etiquetas.append('Visual')
    elif perfil[1] + perfil[4] > 1:
        etiquetas.append('Auditivo')
    elif perfil[2] + perfil[5] > 1:
        etiquetas.append('Kinestésico')
    else:
        etiquetas.append('Mixto')

data = pd.DataFrame(datos, columns=preguntas)
y_map = {'Visual': 0, 'Auditivo': 1, 'Kinestésico': 2, 'Mixto': 3}
y = to_categorical([y_map[e] for e in etiquetas], num_classes=4)
X = data.values

# Paso 3: Modelo TensorFlow
model = tf.keras.Sequential([
    tf.keras.layers.Dense(12, activation='relu', input_shape=(6,)),
    tf.keras.layers.Dense(4, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=150, verbose=0)

# Paso 4: Interfaz
out = Output()
checkboxes = [Checkbox(description=p) for p in preguntas]
boton = Button(description='Detectar estilo')
```

```

rev_map = {v: k for k, v in y_map.items()}

def 40etector(b):
    entrada = np.array([[int(cb.value) for cb in checkboxes]])
    pred = model.predict(entrada)[0]
    estilo = rev_map[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Estilo de aprendizaje dominante: {estilo}")

boton.on_click(40etector)
display(VBox(checkboxes + [boton, out]))

```

Explicación detallada:

PASO 1: Importar bibliotecas

```

import tensorflow as tf
import numpy as np
import pandas as pd
from tensorflow.keras.utils import to_categorical
from ipywidgets import Checkbox, Button, VBox, Output
from IPython.display import display

```

- **tensorflow:** es necesario para crear y entrenar la red neuronal.
- **numpy:** es necesario para generar datos aleatorios.
- **pandas:** es necesario para organizar los datos simulados como una tabla.
- **to_categorical:** es necesario para codificar las etiquetas en formato one-hot.
- **ipywidgets:** es necesario para construir la interfaz interactiva (checkboxes, botón).
- **display:** es necesario para mostrar los widgets en pantalla.

PASO 2: Crear dataset simulado

```

preguntas = ['Prefiero diagramas', 'Escucho podcasts', 'Aprendo
moviéndome',
              'Hago mapas mentales', 'Recuerdo lo que escucho', 'Necesito
tocar']

```

En esta parte del código se crea un dataset simulado.

Primero, cada pregunta representa un indicador del estilo de aprendizaje: Visual → diagramas y mapas mentales, Auditivo → escuchar podcasts y recordar lo oído, Kinestésico → movimiento y manipulación.

```

datos = []
etiquetas = []

```



```

for _ in range(300):
    perfil = np.random.choice([0, 1], size=6)
    datos.append(perfil)
    if perfil[0] + perfil[3] > 1:
        etiquetas.append('Visual')
    elif perfil[1] + perfil[4] > 1:
        etiquetas.append('Auditivo')
    elif perfil[2] + perfil[5] > 1:
        etiquetas.append('Kinestésico')
    else:
        etiquetas.append('Mixto')

```

En esta parte se crean 300 perfiles simulados, cada uno como un arreglo binario de 6 respuestas (1 = sí, 0 = no). Las reglas de asignación determinan el estilo de aprendizaje dominante:

- Visual: si marca más de 1 ítem visual (índice 0 y 3).
- Auditivo: si marca más de 1 ítem auditivo (índice 1 y 4).
- Kinestésico: si marca más de 1 ítem kinestésico (índice 2 y 5).
- Mixto: si no predomina ninguno.

```

data = pd.DataFrame(datos, columns=preguntas)
y_map = {'Visual': 0, 'Auditivo': 1, 'Kinestésico': 2, 'Mixto': 3}
y = to_categorical([y_map[e] for e in etiquetas], num_classes=4)
X = data.values

```

En esta parte, se organiza el dataset en un `DataFrame` y las etiquetas se codifican como vectores para entrenamiento multiclase.

PASO 3: Definir y entrenar el modelo

```

model = tf.keras.Sequential([
    tf.keras.layers.Dense(12, activation='relu', input_shape=(6,)),
    tf.keras.layers.Dense(4, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=150, verbose=0)

```

En esta parte se crea la red neuronal como un modelo secuencial de dos capas:

- Capa oculta: 12 neuronas, activación ReLU.
- Capa de salida: 4 neuronas (una por estilo), activación softmax para clasificación.

Además, se usa `adam` como optimizador y `categorical_crossentropy` como función de pérdida. Por último, se entrena durante 150 épocas.

PASO 4: Interfaz para detección

```

out = Output()
checkboxes = [Checkbox(description=p) for p in preguntas]
boton = Button(description='Detectar estilo')

```

En esta parte, se inicia la creación de la interfaz. Se crea una lista de checkboxes donde el usuario puede indicar sus preferencias. Se añade un botón que ejecuta el análisis.

```

rev_map = {v: k for k, v in y_map.items()}

```

Esta línea del código lo que hace es convertir de número a nombre de estilo.

```

def detector(b):
    entrada = np.array([[int(cb.value) for cb in checkboxes]])
    pred = model.predict(entrada)[0]
    estilo = rev_map[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Estilo de aprendizaje dominante: {estilo}")

```

En esta parte se convierten los valores de los checkboxes en una lista binaria (1 si está marcado, 0 si no). Se hace la predicción con el modelo entrenado. Se extrae el estilo con mayor probabilidad y se muestra el resultado en pantalla.

```

boton.on_click(detector)
display(VBox(checkboxes + [boton, out]))

```

Por último, esta parte hace la conexión con la interfaz. Se conecta el botón a la función `detector` y se organiza todo con `VBox` para mostrarlo ordenadamente en la interfaz.

Actividad práctica 14: “EmoText” Clasificador de emociones en frases de estudiantes

Objetivo: Diseñar una app que identifique emociones básicas (feliz, triste, molesto, neutral, etc) en frases cortas escritas por estudiantes

Alcance: Puede usarse como herramienta de apoyo emocional, ayudando a docentes o tutores a detectar señales tempranas de estados emocionales que requieran atención.

Producto final: App interactiva tipo chatbot donde el usuario escribe una frase y el modelo responde con la emoción detectada.

Código:

```
# Paso 1: Librerías necesarias
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D,
Dense
from ipywidgets import Text, Button, Output, VBox
import numpy as np

# Paso 2: Dataset simulado
frases = [
    ("Me siento muy feliz hoy", "feliz"),
    ("Estoy muy triste por el examen", "triste"),
    ("No entiendo nada y eso me enoja", "molesto"),
    ("Fui a clases y regresé a casa", "neutral"),
    ("Estoy motivado con este curso", "feliz"),
    ("Estoy frustrado por los resultados", "molesto"),
    ("No pasó nada especial hoy", "neutral"),
    ("No me fue bien en el examen", "triste"),
]

texts, labels = zip(*frases)
label_map = {"feliz": 0, "triste": 1, "molesto": 2, "neutral": 3}
y = tf.keras.utils.to_categorical([label_map[label] for label in labels],
num_classes=4)

# Paso 3: Tokenización y padding
tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(texts)
X = tokenizer.texts_to_sequences(texts)
X = pad_sequences(X, padding='post', maxlen=10)

# Paso 4: Modelo
model = Sequential([
    Embedding(1000, 16, input_length=10),
    GlobalAveragePooling1D(),
    Dense(16, activation='relu'),
    Dense(4, activation='softmax')
])

model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
model.fit(X, y, epochs=200, verbose=0)

# Paso 5: Interfaz interactiva
out = Output()
entrada = Text(description="Frase:")
boton = Button(description="Detectar Emoción")
```

```

rev_map = {0: "Feliz", 1: "Triste", 2: "Molesto", 3: "Neutral"}

def detectar_emocion(b):
    seq = tokenizer.texts_to_sequences([entrada.value])
    pad = pad_sequences(seq, padding='post', maxlen=10)
    pred = model.predict(pad)[0]
    emocion = rev_map[np.argmax(pred)]
    with out:
        out.clear_output()
        print(f"Emoción detectada: {emocion}")

boton.on_click(detectar_emocion)

VBox([entrada, boton, out])

```

Actividad práctica 15: “RecomEdu” Recomendador de recursos educativos personalizados

Objetivo: Diseñar una app interactiva que, a partir de los resultados de una autoevaluación diagnóstica sencilla, recomiende recursos educativos adaptados al nivel del estudiante (bajo, medio, alto).

Alcance: Puede usarse al inicio de un curso para ofrecer rutas de aprendizaje adaptativas a estudiantes con distintos niveles previos de conocimiento.

Producto final: App interactiva con controles deslizantes para responder preguntas diagnósticas, que muestra recomendaciones de recursos al usuario con base a su nivel estimado.

Código:

```
# Paso 1: Importar bibliotecas necesarias
import tensorflow as tf
import numpy as np
import pandas as pd
from ipywidgets import IntSlider, Button, VBox, Output

# Paso 2: Simular datos de entrada
data = pd.DataFrame({
    'preg1': np.random.randint(0, 2, 200),
    'preg2': np.random.randint(0, 2, 200),
    'preg3': np.random.randint(0, 2, 200)
})
data['total'] = data.sum(axis=1)

# Etiquetas (0 = bajo, 1 = medio, 2 = alto)
data['nivel'] = data['total'].apply(lambda x: 0 if x == 0 else (1 if x == 1 or x == 2 else 2))
X = data[['preg1', 'preg2', 'preg3']].values
y = tf.keras.utils.to_categorical(data['nivel'], num_classes=3)

# Paso 3: Crear modelo
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation='relu', input_shape=(3,)),
    tf.keras.layers.Dense(3, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=100, verbose=0)

# Paso 4: Interfaz interactiva
out = Output()
s1 = IntSlider(min=0, max=1, description='Pregunta 1')
s2 = IntSlider(min=0, max=1, description='Pregunta 2')
s3 = IntSlider(min=0, max=1, description='Pregunta 3')
boton = Button(description='Recomendar')

recursos = {
    0: "Recomendamos repasar conceptos básicos: [Video
Introdutorio] (https://www.khanacademy.org)",
    1: "Buen comienzo. Recomendamos: [Lectura Guiada + Ejercicios
Interactivos] (https://www.edx.org)",
    2: "¡Excelente! Avanza con: [Proyecto
Avanzado] (https://www.coursera.org)"
}

def recomendar(b):
    entrada = np.array([s1.value, s2.value, s3.value])
    pred = model.predict(entrada)[0]
    nivel = np.argmax(pred)
    with out:
```

```
out.clear_output()
print(f"Nivel estimado: {'Bajo', 'Medio', 'Alto'}[nivel]")
print(recursos[nivel])

boton.on_click(recomendar)
VBox([s1, s2, s3, boton, out])
```

Actividad práctica 16: “MathCheck” Detector de errores comunes en expresiones algebraicas

Objetivo: Diseñar una app educativa que permita a los usuarios ingresar expresiones algebraicas y detectar automáticamente si contiene errores típicos (como errores de factorización o aplicación incorrecta de propiedades algebraicas).

Alcance: Esta app ayuda a los docentes a identificar errores frecuentes en el aprendizaje del álgebra básica y puede usarse como herramienta de

retroalimentación inmediata en actividades con estudiantes de secundaria o preparatoria.

Producto final: una app simple con un cuadro de texto donde el estudiante ingresa su expresión y el sistema responde si es correcta o contiene errores.

Código:

```
# Paso 1: Librerías necesarias
import tensorflow as tf
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical
from ipywidgets import Text, Button, Output, VBox

# Paso 2: Simulación de datos de entrenamiento
expresiones = [
    "(x + 2) (x + 3) = x^2 + 5x + 6",
    "x(x + 4) = x^2 + 4x",
    "(x - 1)^2 = x^2 - 2x + 1",
    "(x + 2) (x + 3) = x^2 + 6x + 5",
    "x(x + 4) = x^2 + 5x",
    "(x - 1)^2 = x^2 - 1"
]
etiquetas = ["correcto", "correcto", "correcto", "error", "error",
"error"]

# Preprocesamiento
le = LabelEncoder()
labels = le.fit_transform(etiquetas)
labels_cat = to_categorical(labels)

# Convertir expresiones a vectores simples de caracteres (one-hot
rudimentario)
from sklearn.feature_extraction.text import CountVectorizer
vec = CountVectorizer(binary=True, analyzer='char', max_features=40)
X = vec.fit_transform(expresiones).toarray()

# Paso 3: Modelo
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X.shape[1],)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(2, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, labels_cat, epochs=50, verbose=0)

# Paso 4: Interfaz
entrada = Text(description='Expresión:')
boton = Button(description='Verificar')
out = Output()

def verificar(b):
    texto = entrada.value
    vector = vec.transform([texto]).toarray()
    pred = model.predict(vector)
```

```
resultado = le.inverse_transform([np.argmax(pred)])[0]
with out:
    out.clear_output()
    print(f"Resultado: {resultado.upper()}")

boton.on_click(verificar)
VBox([entrada, boton, out])
```

Actividad práctica 17: “VocabTrainer” App para entrenar y evaluar vocabulario en inglés mediante reconocimiento de voz

Objetivo: Crear una app educativa que ayude a los usuarios a practicar y mejorar su vocabulario en inglés a través de ejercicios de reconocimiento de voz

Alcance: permite al usuario escuchar una palabra en inglés, intentar pronunciarla y recibir retroalimentación inmediata sobre si su pronunciación fue correcta o necesita mejorar, fomentando la práctica oral.

Código:


```

# Importar librerías necesarias
import tensorflow as tf
import numpy as np
import IPython.display as ipd
import speech_recognition as sr
from ipywidgets import Button, Output, VBox

# Nota: Para usar reconocimiento de voz en Google Colab se puede usar la
biblioteca SpeechRecognition junto con el micrófono local.

# Lista de palabras para practicar
palabras = ["apple", "banana", "cat", "dog", "elephant"]

out = Output()

def escuchar_y_validar(palabra_objetivo):
    r = sr.Recognizer()
    with sr.Microphone() as source:
        with out:
            out.clear_output()
            print(f"Por favor, pronuncia la palabra: {palabra_objetivo}")
            audio = r.listen(source)
    try:
        texto = r.recognize_google(audio, language='en-US')
        with out:
            if texto.lower() == palabra_objetivo.lower():
                print(f";Correcto! Dijiste: {texto}")
            else:
                print(f"Escuché: {texto}. Intenta nuevamente.")
    except Exception as e:
        with out:
            print(f"Error al reconocer la voz: {e}")

btns = []
for palabra in palabras:
    btn = Button(description=palabra)
    btn.on_click(lambda b, p=palabra: escuchar_y_validar(p))
    btns.append(btn)

VBox(btns + [out])

```

Actividad práctica 18: “MathQuiz” Generador y evaluador de preguntas de Matemáticas con detección automática de respuestas

Objetivo: Diseñar una app educativa que genere preguntas básicas de matemáticas (suma, resta, multiplicación) y evalúe automáticamente la respuesta del usuario para validar respuestas y detectar patrones de errores comunes.

Alcance: la app ayuda a estudiantes de primaria a practicar operaciones matemáticas y permite a docentes identificar dificultades recurrentes mediante análisis automático de respuestas.

Código:

```
import tensorflow as tf
import numpy as np
import random
from ipywidgets import interact_manual, IntText, Output, VBox

out = Output()
# Generador de preguntas
def generar_pregunta():
    operadores = ['+', '-', '*']
    a = random.randint(1, 20)
    b = random.randint(1, 20)
    op = random.choice(operadores)
    pregunta = f"{a} {op} {b}"
    respuesta_correcta = eval(pregunta)
    return pregunta, respuesta_correcta

# Modelo simple para detectar si respuesta es correcta (0 o 1)
# En este caso, no es necesario un modelo complejo, pero se simula para
fines didácticos
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(1,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Datos de entrenamiento simples: respuestas correctas (1) y algunas
erróneas (0)
X_train = np.array([0,1,2,3,4,5,6,7,8,9,10])
y_train = np.array([1,1,1,1,0,0,1,0,1,1,0])
model.fit(X_train, y_train, epochs=10, verbose=0)
def evaluar_respuesta(respuesta_usuario, respuesta_correcta):
    entrada = np.array([respuesta_usuario - respuesta_correcta])
    prediccion = model.predict(entrada.reshape(-1,1))[0][0]
    return prediccion > 0.5
def quiz():
    pregunta, correcta = generar_pregunta()
    with out:
        out.clear_output()
        print(f"Pregunta: ¿Cuál es el resultado de {pregunta}?")
    def evaluar(respuesta_usuario):
        correcto = evaluar_respuesta(respuesta_usuario, correcta)
        with out:
            if correcto:
                print("¡Respuesta correcta!")
            else:
                print(f"Respuesta incorrecta. La respuesta correcta es
{correcta}")

    interact_manual(evaluar, respuesta_usuario=IntText(description="Tu
respuesta:"))

VBox([out])
```

Actividad práctica 19: “VocabTrainer” App para reconocimiento y clasificación básica de palabras en inglés con aprendizaje supervisado

Objetivo: Diseñar una app educativa que permita a los usuarios a aprender vocabulario en inglés clasificando palabras en categorías (por ejemplo: animal, objeto, comida, etc) mediante un modelo de red neuronal simple.

Alcance: La app sirve para apoyar el aprendizaje de vocabulario básico y su clasificación en categorías, fomentando la memorización y la familiarización con palabras comunes en inglés.

Código:

```
import tensorflow as tf
```

```

import numpy as np
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from ipywidgets import Dropdown, Button, Output, VBox, Label

# Dataset simple con palabras y categorías
palabras = [
    'dog', 'cat', 'apple', 'banana', 'car', 'bus', 'lion', 'tiger',
    'bread', 'milk'
]
categorias = [
    'animal', 'animal', 'food', 'food', 'vehicle', 'vehicle', 'animal',
    'animal', 'food', 'food'
]

# Codificar categorías
label_encoder = LabelEncoder()
labels_encoded = label_encoder.fit_transform(categorias)

# Tokenizar palabras
tokenizer = Tokenizer(char_level=True)
tokenizer.fit_on_texts(palabras)
sequences = tokenizer.texts_to_sequences(palabras)
max_len = max(len(seq) for seq in sequences)
X = pad_sequences(sequences, maxlen=max_len, padding='post')

# Definir modelo
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=len(tokenizer.word_index) + 1,
    output_dim=8, input_length=max_len),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10, activation='relu'),
    tf.keras.layers.Dense(len(set(labels_encoded)), activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.fit(X, labels_encoded, epochs=100, verbose=0)

# Interfaz para probar palabras nuevas
out = Output()
dropdown = Dropdown(options=palabras, description='Palabra:')
def clasificar_palabra(change):
    palabra = change['new']
    seq = tokenizer.texts_to_sequences([palabra])
    seq_pad = pad_sequences(seq, maxlen=max_len, padding='post')
    pred = model.predict(seq_pad)
    categoria_pred = label_encoder.inverse_transform([np.argmax(pred)])
    with out:
        out.clear_output()
        print(f"La palabra '{palabra}' pertenece a la categoría:
{categoria_pred[0]}")
dropdown.observe(clasificar_palabra, names='value')
Label("Selecciona una palabra para clasificar:"),
VBox([dropdown, out])

```

Actividad práctica 20: Chatbot para resolver ecuaciones lineales simples

Objetivo: Apoyar al estudiante a practicar y resolver ecuaciones lineales básicas y validar su solución.

Alcance: Generación aleatoria de ejercicios, evaluación automática de la respuesta del usuario, retroalimentación inmediata e interfaz interactiva con botones y entrada de texto.

Producto final: un notebook funcional en Google Colab con interfaz visual donde el estudiante resuelve ecuaciones aleatorias, recibe evaluación automática y accede a la solución paso a paso.

Código:

```

import ipywidgets as widgets
from IPython.display import display, clear_output
import random
from sympy import Symbol, Eq, solve, simplify

x = Symbol('x')

# Genera una ecuación aleatoria del tipo  $ax + b = c$ 
def generar_ecuacion():
    a = random.randint(1, 10)
    b = random.randint(-5, 5)
    c = random.randint(0, 20)
    eq = Eq(a * x + b, c)
    sol = solve(eq, x)[0]
    return eq, sol, f"Paso 1: Resta {b} a ambos lados.\nPaso 2: Divide
entre {a}.\nResultado:  $x = {sol}$ "

# Widgets
entrada = widgets.Text(placeholder='Ingresa el valor de x')
boton = widgets.Button(description='Verificar')
boton_pasos = widgets.Button(description='Ver pasos')
salida = widgets.Output()

# Estado
ecuacion, solucion, pasos = generar_ecuacion()

def mostrar_ejercicio():
    salida.clear_output()
    entrada.value = ""
    with salida:
        print("Resuelve la siguiente ecuación:")
        print(ecuacion)

def verificar(b):
    try:
        respuesta = float(entrada.value)
        if simplify(respuesta - solucion) == 0:
            msg = "✓ ¡Correcto!"
        else:
            msg = f"✗ Incorrecto. La respuesta correcta no es
{respuesta}"
    except:
        msg = "☐ Ingresa un número válido."
    with salida:
        print(msg)

def mostrar_pasos(b):
    with salida:
        print("☐ Pasos para resolver:")
        print(pasos)

boton.on_click(verificar)
boton_pasos.on_click(mostrar_pasos)

display(salida, entrada, widgets.HBox([boton, boton_pasos]))
mostrar_ejercicio()

```

Actividad práctica 21: Chatbot para interpretar gráficos estadísticos

Objetivo: Apoyar al estudiante a interpretar gráficos estadísticos básicos de barra o de pastel, identificando tendencias o datos claves.

Alcance: Generación del gráfico con matplotlib, pregunta relacionada con el gráfico y retroalimentación automática sobre la respuesta del usuario.

Producto final: una app interactiva en Google Colab que muestra un gráfico estadístico, pregunta algo sobre él y valida la respuesta del estudiante.

Código:

```
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output
```

```

# Datos
categorias = ['A', 'B', 'C', 'D']
valores = [5, 8, 6, 3]
respuesta_correcta = 'B'

# Gráfico
def graficar():
    plt.figure(figsize=(6,4))
    plt.bar(categorias, valores, color='skyblue')
    plt.title("Estudiantes por grupo")
    plt.xlabel("Grupo")
    plt.ylabel("Cantidad")
    plt.show()

# Widgets
entrada = widgets.Text(placeholder="¿Qué grupo tiene más estudiantes?")
boton = widgets.Button(description='Verificar')
salida = widgets.Output()

def verificar(b):
    r = entrada.value.strip().upper()
    with salida:
        clear_output()
        graficar()
        if r == respuesta_correcta:
            print("✓ ¡Correcto! El grupo B tiene 8 estudiantes.")
        else:
            print(f"✗ Incorrecto. La respuesta correcta es '{respuesta_correcta}'.")

# Mostrar interfaz
with salida:
    graficar()
display(salida, entrada, boton)
boton.on_click(verificar)

```

Explicación detallada:

Paso 1. Importación de bibliotecas

```

import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output

```

En esta parte se importan todas las bibliotecas necesarias:

- `matplotlib.pyplot as plt`: biblioteca para generar gráficos, aquí se usa para hacer un gráfico de barras.

- `ipywidgets`: permite crear elementos interactivos como cajas de texto y botones.
- `display`, `clear_output`: funciones de Jupyter/Colab para mostrar y limpiar contenido dinámicamente.

Paso 2. Definición de datos

```
categorias = ['A', 'B', 'C', 'D']
valores = [5, 8, 6, 3]
respuesta_correcta = 'B'
```

En esta parte, se definen los datos:

- `categorias`: son las etiquetas para los grupos de estudiantes.
- `valores`: es el número de estudiantes en cada grupo.
- `respuesta_correcta`: el grupo con mayor número de estudiantes (grupo B, con 8).

Paso 3. Función para graficar

```
def graficar():
    plt.figure(figsize=(6,4))
    plt.bar(categorias, valores, color='skyblue')
    plt.title("Estudiantes por grupo")
    plt.xlabel("Grupo")
    plt.ylabel("Cantidad")
    plt.show()
```

En esta parte se define la función para graficar. Esta función: separa la parte visual del resto del código y hace que pueda llamarse varias veces, por ejemplo, al inicio y luego para dar retroalimentación.

Paso 4. Creación de widgets (interfaz)

```
entrada = widgets.Text(placeholder="¿Qué grupo tiene más estudiantes?")
boton = widgets.Button(description='Verificar')
salida = widgets.Output()
```

En esta parte se crea una interfaz muy sencilla que contiene la `entrada` la cual representa la caja de texto donde el usuario escribe su respuesta, `boton` que representa el botón con la etiqueta "Verificar" y la `salida` el cual es el contenedor visual donde se mostrará el gráfico y el resultado.

Paso 5. Lógica de verificación

```
def verificar(b):
    r = entrada.value.strip().upper()
    with salida:
        clear_output()
        graficar()
```

```

    if r == respuesta_correcta:
        print("✓ ;Correcto! El grupo B tiene 8 estudiantes.")
    else:
        print(f"✗ Incorrecto. La respuesta correcta es
'{respuesta_correcta}'.")

```

En esta parte del código se define la lógica de verificación del modelo por medio de la siguiente estructura:

- `def verificar(b):` función que se ejecuta cuando se hace clic en el botón.
- `r = entrada.value.strip().upper():`
 - `Obtiene el valor de la caja de texto.`
 - `strip():` quita espacios.
 - `upper():` convierte a mayúsculas para evitar errores por minúsculas.
- `with salida:` todo lo que se imprime aquí aparece dentro del widget `salida`.
- `clear_output():` limpia contenido anterior.
- `graficar():` se llama nuevamente para volver a mostrar el gráfico.
- Condicional `if:` compara la respuesta ingresada con la correcta.
 - Muestra mensaje de acierto o error según corresponda.

Paso 6. Mostrar la interfaz completa

```

with salida:
    graficar()
display(salida, entrada, boton)
boton.on_click(verificar)

```

Esta parte del código muestra la interfaz completa con la siguiente estructura:

- `with salida: graficar():` muestra el gráfico la primera vez antes de responder.
- `display(...):` muestra la salida visual, la caja de texto y el botón.
- `boton.on_click(verificar):` conecta el botón con la función `verificar`, de forma que cada clic active esa función.

Actividad práctica 22: Chatbot sobre cálculo de áreas de figuras geométricas

Objetivo: Practicar el cálculo de áreas de figuras geométricas con retroalimentación automatizada en tiempo real, registre el desempeño del estudiante, ajuste la dificultad y guíe paso a paso de la solución.

Alcance: solo para figuras geométricas básicas, tales como: triángulo, cuadrado, rectángulo, círculo y trapecio.

Producto final: una app interactiva en Google Colab con una interfaz visual, que verifique los resultados de forma automática, explique paso a paso la solución y genere infinitos ejercicios con diferentes valores.

Código:

```

import ipywidgets as widgets
from IPython.display import display, clear_output
import random, math, time

salida = widgets.Output()

# Figuras por nivel

def generar_triangulo():
    base = random.randint(4, 10)
    altura = random.randint(3, 8)
    area = round((base * altura) / 2, 2)
    pasos = f"Área = (base * altura) / 2 = ({base} * {altura}) / 2 = {area}"
    return "Triángulo", f"Base = {base}, Altura = {altura}", area, pasos

def generar_cuadrado():
    lado = random.randint(3, 10)
    area = round(lado ** 2, 2)
    pasos = f"Área = lado2 = {lado}2 = {area}"
    return "Cuadrado", f"Lado = {lado}", area, pasos

def generar_rectangulo():
    base = random.randint(4, 10)
    altura = random.randint(3, 8)
    area = round(base * altura, 2)
    pasos = f"Área = base * altura = {base} * {altura} = {area}"
    return "Rectángulo", f"Base = {base}, Altura = {altura}", area, pasos

def generar_circulo():
    radio = random.randint(2, 6)
    area = round(math.pi * radio ** 2, 2)
    pasos = f"Área =  $\pi * r^2 = \pi * \{radio\}^2 = \pi * \{radio**2\} \approx \{area\}"
    return "Círculo", f"Radio = {radio}", area, pasos

def generar_trapezio():
    base1 = random.randint(3, 7)
    base2 = random.randint(4, 9)
    altura = random.randint(3, 6)
    area = round(((base1 + base2) * altura) / 2, 2)
    pasos = f"Área = ((base1 + base2) * altura) / 2 = (({base1} + {base2}) * {altura}) / 2 = {area}"
    return "Trapezio", f"Base1 = {base1}, Base2 = {base2}, Altura = {altura}", area, pasos

# Niveles
niveles = {
    1: [generar_triangulo, generar_cuadrado],
    2: [generar_rectangulo],
    3: [generar_circulo, generar_trapezio]
}

# Variables globales

nivel = 1
aciertos_consecutivos = 0
puntuacion = 0$ 
```

```

tiempos = []
ejercicio_actual = {}
inicio_tiempo = 0

# Widgets

entrada_area = widgets.Text(placeholder="Ingresa el área")
btn_verificar = widgets.Button(description="✓ Verificar")
btn_pasos = widgets.Button(description="□ Ver pasos")
btn_siguiente = widgets.Button(description="□ Siguiente")
lbl_estado = widgets.Label()

# Lógica principal

def nuevo_ejercicio(_=None):
    global ejercicio_actual, inicio_tiempo
    gen = random.choice(niveles[nivel])
    nombre, datos, area, pasos = gen()
    ejercicio_actual = {
        "nombre": nombre,
        "datos": datos,
        "area": area,
        "pasos": pasos
    }
    entrada_area.value = ""
    inicio_tiempo = time.time()
    salida.clear_output()
    with salida:
        print(f"□ Figura: {nombre}")
        print(f"□ Datos: {datos}")

def verificar(_):
    global puntuacion, aciertos_consecutivos, nivel
    try:
        respuesta = float(entrada_area.value)
        tiempo = round(time.time() - inicio_tiempo, 2)
        tiempos.append(tiempo)
        salida.clear_output()
        with salida:
            print(f"□ Figura: {ejercicio_actual['nombre']}")
            print(f"□ Datos: {ejercicio_actual['datos']}")
            print(f"□ Tiempo de respuesta: {tiempo} s")
            if abs(respuesta - ejercicio_actual['area']) < 0.5:
                puntuacion += 10
                aciertos_consecutivos += 1
                print("✓ ¡Correcto!")
                if aciertos_consecutivos >= 3 and nivel < 3:
                    nivel += 1
                    print(f"□ Nivel aumentado a {nivel}")
                    aciertos_consecutivos = 0
            else:
                aciertos_consecutivos = 0
                print(f"✗ Incorrecto. El área correcta es ≈ {ejercicio_actual['area']}")
            lbl_estado.value = f"□ Puntos: {puntuacion} | Nivel: {nivel} | Tiempo promedio: {round(sum(tiempos)/len(tiempos), 2)} s"

```

```

    except:
        with salida:
            print("❌ Ingresa un número válido.")

def mostrar_pasos(_):
    with salida:
        print("❌ Pasos para resolver:")
        print(ejercicio_actual['pasos'])

# Conexión de eventos

btn_verificar.on_click(verificar)
btn_pasos.on_click(mostrar_pasos)
btn_siguiente.on_click(nuevo_ejercicio)

# Interfaz

display(salida)
display(entrada_area)
display(widgets.HBox([btn_verificar, btn_pasos, btn_siguiente]))
display(lbl_estado)

nuevo_ejercicio()

```

Explicación detallada:

Paso 1. Importación de bibliotecas

```

import ipywidgets as widgets
from IPython.display import display, clear_output
import random, math, time

```

En esta parte se importan todas las bibliotecas necesarias:

- **ipywidgets:** permite crear elementos de interfaz como botones, entradas, etiquetas.
- **display, clear_output:** funciones para mostrar elementos dinámicamente en celdas del notebook y limpiar salidas anteriores.
- **random:** para generar valores aleatorios (dimensiones de figuras).
- **math:** se usa para π en el área del círculo.
- **time:** mide el tiempo que tarda el estudiante en responder.

Paso 2. Contenedor de salida

```

salida = widgets.Output()

```

Este widget es el contenedor visual donde se mostrarán las figuras, resultados, mensajes, pasos, etc.

Paso 3. Funciones generadoras de figuras

```
def generar_triangulo():
    base = random.randint(4, 10)
    altura = random.randint(3, 8)
    area = round((base * altura) / 2, 2)
    pasos = f"Área = (base * altura) / 2 = ({base} * {altura}) / 2 = {area}"
    return "Triángulo", f"Base = {base}, Altura = {altura}", area, pasos
```

Cada función genera una figura geométrica distinta, con valores aleatorios y su área calculada.

- `base` y `altura` se generan aleatoriamente.
- `area` se calcula con la fórmula del área de un triángulo.
- `round(..., 2)` limita el resultado a 2 decimales.
- `pasos`: cadena de texto que explica cómo se llegó al resultado.
- `return`: nombre de la figura, datos visibles, área numérica y pasos explicativos.

Las funciones para el cuadrado, el rectángulo, el círculo y el trapecio siguen el mismo patrón, con sus fórmulas correspondientes:

- Cuadrado: $A = lado^2$
- Rectángulo: $A = base \times altura$
- Círculo: $A = \pi r^2$
- Trapecio: $A = \frac{(b_1 + b_2)h}{2}$

Paso 4. Organización por niveles

```
niveles = {
    1: [generar_triangulo, generar_cuadrado],
    2: [generar_rectangulo],
    3: [generar_circulo, generar_trapecio]
}
```

En esta parte del código se organizan los niveles de dificultad con listas de funciones generadoras. A medida que el usuario progresa, accede a figuras más complejas.

Paso 5. Variables globales

```
nivel = 1
aciertos_consecutivos = 0
puntuación = 0
tiempos = []
ejercicio_actual = {}
inicio_tiempo = 0
```

En esta parte, se definen las variables globales:

- `nivel`: nivel actual del jugador.
- `aciertos_consecutivos`: si acierta 3 veces seguidas, sube de nivel.
- `puntuacion`: puntos acumulados (10 por cada acierto).
- `tiempos`: lista para almacenar duración de cada intento.
- `ejercicio_actual`: almacena la figura actual.
- `inicio_tiempo`: marca de tiempo en que empieza el ejercicio.

Paso 6. Widgets para la interfaz

```
entrada_area = widgets.Text(placeholder="Ingresa el área")
btn_verificar = widgets.Button(description="✓ Verificar")
btn_pasos = widgets.Button(description="□ Ver pasos")
btn_siguiente = widgets.Button(description="□ Siguiente")
lbl_estado = widgets.Label()
```

En esta parte, se crea los widgets necesarios para la interfaz con la siguiente estructura:

- `entrada_area`: campo para que el estudiante escriba su respuesta.
- `btn_verificar`: botón para validar la respuesta.
- `btn_pasos`: muestra la solución paso a paso.
- `btn_siguiente`: genera un nuevo ejercicio.
- `lbl_estado`: etiqueta que muestra puntuación, nivel y tiempo promedio.

Paso 7. Función para crear un nuevo ejercicio

```
def nuevo_ejercicio(_=None):
    global ejercicio_actual, inicio_tiempo
    gen = random.choice(niveles[nivel])
    nombre, datos, area, pasos = gen()
    ejercicio_actual = {
        "nombre": nombre,
        "datos": datos,
        "area": area,
        "pasos": pasos
    }
    entrada_area.value = ""
    inicio_tiempo = time.time()
    salida.clear_output()
    with salida:
        print(f"□ Figura: {nombre}")
        print(f"□ Datos: {datos}")
```

En esta parte del código se define un nuevo ejercicio. Se escoge aleatoriamente una figura según el nivel actual. Se guardan los datos generados en `ejercicio_actual`. Se limpia el campo de entrada. Se reinicia el cronómetro (`inicio_tiempo`) y se muestra la figura y sus datos al usuario.

Paso 8. Función de verificación

```
def verificar(_):
    global puntuacion, aciertos_consecutivos, nivel
    try:
        respuesta = float(entrada_area.value)
        tiempo = round(time.time() - inicio_tiempo, 2)
        tiempos.append(tiempo)
        salida.clear_output()
        with salida:
            print(f"□ Figura: {ejercicio_actual['nombre']}")
            print(f"□ Datos: {ejercicio_actual['datos']}")
            print(f"□ Tiempo de respuesta: {tiempo} s")
            if abs(respuesta - ejercicio_actual['area']) < 0.5:
                puntuacion += 10
                aciertos_consecutivos += 1
                print("✓ ¡Correcto!")
                if aciertos_consecutivos >= 3 and nivel < 3:
                    nivel += 1
                    print(f"□ Nivel aumentado a {nivel}")
                    aciertos_consecutivos = 0
            else:
                aciertos_consecutivos = 0
                print(f"✗ Incorrecto. El área correcta es ~
{ejercicio_actual['area']}")
                lbl_estado.value = f"□ Puntos: {puntuacion} | Nivel: {nivel}
| Tiempo promedio: {round(sum(tiempos)/len(tiempos), 2)} s"
    except:
        with salida:
            print("□ Ingresa un número válido.")
```

En esta parte, se define la función de verificación que:

- Intenta convertir el valor ingresado a `float`.
- Calcula el tiempo de respuesta y lo guarda.
- Compara la respuesta del estudiante con el valor correcto (± 0.5 de tolerancia).
- Si es correcta:
 - Suma 10 puntos.
 - Aumenta aciertos consecutivos.

- Si llega a 3 aciertos y aún no está en el nivel 3, sube de nivel.
- Si es incorrecta:
 - Reinicia los aciertos consecutivos.
 - Muestra la respuesta correcta.
- Actualiza la `lbl_estado` con la puntuación y promedio de tiempo.

También maneja errores (por ejemplo, si el usuario escribe letras) con un mensaje claro.

Paso 9. Mostrar pasos de resolución

```
def mostrar_pasos(_):
    with salida:
        print("□ Pasos para resolver:")
        print(ejercicio_actual['pasos'])
```

En esta parte, se imprime la explicación guardada sobre cómo se resolvió el área.

Paso 10. Conexión de botones a funciones

```
btn_verificar.on_click(verificar)
btn_pasos.on_click(mostrar_pasos)
btn_siguiente.on_click(nuevo_ejercicio)
```

En esta parte se conectan los botones con sus funciones respectivas.

Paso 11. Renderizado de la interfaz

```
display(salida)
display(entrada_area)
display(widgets.HBox([btn_verificar, btn_pasos, btn_siguiente]))
display(lbl_estado)
```

En esta parte del código se muestra el renderizado de la interfaz. Se muestran todos los elementos visuales en el orden adecuado y `HBox` agrupa los tres botones en una fila.

Paso 12. Inicio automático del primer ejercicio

```
nuevo_ejercicio()
```

Por último, esta línea lo que hace es llamar a la función `nuevo_ejercicio()` al cargar el programa para empezar de inmediato.

Como conclusión para los docentes, este código:

- Promueve el aprendizaje activo mediante ejercicios de geometría.

- Incorpora un poco de gamificación (puntos, niveles, tiempo).
- Permite el desarrollo de competencias lógico-matemáticas y atención al detalle.
- Puede ser fácilmente ampliado para incluir figuras nuevas o exportar resultados.
- Es ideal para usarse en clases de matemáticas, ingeniería o ciencias.

Actividad práctica 23: Chatbot de análisis vocacional

Objetivo: Guiar al estudiante en la exploración de su perfil vocacional mediante un test interactivo simple con retroalimentación inmediata.

Alcance: uso de ipywidgets para preguntas dinámicas, registro de respuestas en tiempo real, clasificación automática en 4 perfiles (científico, técnico, artístico, social), retroalimentación detallada y amigable.

Producto final: una app interactiva que presente una pregunta a la vez, al final muestre el perfil vocacional predominante y recomendaciones, opción de volver a empezar.

Código:

```
import ipywidgets as widgets
from IPython.display import display, clear_output

# Preguntas y respuestas
preguntas = [
    ("¿Qué actividad disfrutas más?", ["Leer artículos científicos",
    "Pintar o dibujar", "Reparar cosas", "Ayudar a personas"]),
```

```

    ("¿Qué asignatura prefieres?", ["Matemáticas", "Arte", "Tecnología",
"Psicología"]),
    ("¿Cómo te gusta trabajar?", ["Analizando datos", "Creando cosas
nuevas", "Con herramientas", "En equipo con personas"]),
    ("¿Qué hobby te interesa más?", ["Resolver acertijos", "Tocar un
instrumento", "Armar circuitos", "Voluntariado"]),
    ("¿Qué valoras más en un trabajo?", ["Descubrimiento", "Expresión",
"Precisión", "Impacto social"]),
    ("¿Con qué palabra te identificas más?", ["Lógico", "Creativo",
"Práctico", "Empático"])
]

```

```

# Correspondencia para perfiles

```

```

perfil_map = {
    "Leer artículos científicos": "científico",
    "Matemáticas": "científico",
    "Analizando datos": "científico",
    "Resolver acertijos": "científico",
    "Descubrimiento": "científico",
    "Lógico": "científico",

    "Pintar o dibujar": "artístico",
    "Arte": "artístico",
    "Creando cosas nuevas": "artístico",
    "Tocar un instrumento": "artístico",
    "Expresión": "artístico",
    "Creativo": "artístico",

    "Reparar cosas": "técnico",
    "Tecnología": "técnico",
    "Con herramientas": "técnico",
    "Armar circuitos": "técnico",
    "Precisión": "técnico",
    "Práctico": "técnico",

    "Ayudar a personas": "social",
    "Psicología": "social",
    "En equipo con personas": "social",
    "Voluntariado": "social",
    "Impacto social": "social",
    "Empático": "social"
}

```

```

# Resultados y recomendaciones

```

```

recomendaciones = {
    "científico": "☐ Perfil Científico: Podrías destacar en áreas como
Física, Matemáticas, Biología, Ingeniería o Investigación.",
    "artístico": "☐ Perfil Artístico: Podrías sobresalir en Diseño,
Música, Artes Visuales, Publicidad o Cine.",
    "técnico": "☐ Perfil Técnico: Carreras como Mecatrónica, Robótica,
Sistemas o Mantenimiento son una excelente opción.",
    "social": "☐ Perfil Social: Psicología, Educación, Trabajo Social o
Comunicación podrían ser tu vocación."
}

```

```

# Control de flujo
indice_pregunta = 0

```

```

respuestas = []
conteo = {"científico": 0, "artístico": 0, "técnico": 0, "social": 0}

# Widgets
label_pregunta = widgets.Label()
radio_respuesta = widgets.RadioButtons()
boton_siguiente = widgets.Button(description="Siguiente")
barra = widgets.IntProgress(min=0, max=len(preguntas), value=0)
salida = widgets.Output()

def mostrar_pregunta():
    label_pregunta.value = preguntas[indice_pregunta][0]
    radio_respuesta.options = preguntas[indice_pregunta][1]
    barra.value = indice_pregunta

def siguiente(_):
    global indice_pregunta
    seleccion = radio_respuesta.value
    respuestas.append(seleccion)
    perfil = perfil_map.get(seleccion)
    if perfil:
        conteo[perfil] += 1

    indice_pregunta += 1
    if indice_pregunta < len(preguntas):
        mostrar_pregunta()
    else:
        mostrar_resultado()

def mostrar_resultado():
    salida.clear_output()
    with salida:
        print("✓ Test completado.\n")
        perfil_final = max(conteo, key=conteo.get)
        print(f"□ Tu perfil vocacional dominante es:
**{perfil_final.upper()}**\n")
        print(recomendaciones[perfil_final])
        print("\nRespuestas seleccionadas:")
        for i, r in enumerate(respuestas):
            print(f"{i+1}. {preguntas[i][0]} → {r}")
        boton_siguiente.disabled = True

# Botón reinicio (opcional)
def reiniciar(_):
    global indice_pregunta, respuestas, conteo
    indice_pregunta = 0
    respuestas = []
    conteo = {k: 0 for k in conteo}
    boton_siguiente.disabled = False
    salida.clear_output()
    mostrar_pregunta()

boton_reiniciar = widgets.Button(description="□ Reiniciar")
boton_reiniciar.on_click(reiniciar)

# Conectar eventos
boton_siguiente.on_click(siguiente)

```

```
# Mostrar
mostrar_pregunta()
display(barra, label_pregunta, radio_respuesta, boton_siguiete, salida,
boton_reiniciar)
```

Explicación detallada:

Paso 1. Importación de bibliotecas

```
import ipywidgets as widgets
from IPython.display import display, clear_output
```

En esta parte se importan las bibliotecas necesarias:

- `ipywidgets`: biblioteca que permite crear interfaces gráficas interactivas dentro de notebooks, como botones, barras de progreso, y preguntas tipo radio.
- `display`: función para mostrar widgets y elementos en pantalla.
- `clear_output`: limpia el contenido previamente mostrado, útil al actualizar preguntas o resultados.

Paso 2. Definición del cuestionario

```
preguntas = [
    ("¿Qué actividad disfrutas más?", ["Leer artículos científicos",
    "Pintar o dibujar", "Reparar cosas", "Ayudar a personas"]),
    ("¿Qué asignatura prefieres?", ["Matemáticas", "Arte", "Tecnología",
    "Psicología"]),
    ("¿Cómo te gusta trabajar?", ["Analizando datos", "Creando cosas
nuevas", "Con herramientas", "En equipo con personas"]),
    ("¿Qué hobby te interesa más?", ["Resolver acertijos", "Tocar un
instrumento", "Armar circuitos", "Voluntariado"]),
    ("¿Qué valoras más en un trabajo?", ["Descubrimiento", "Expresión",
    "Precisión", "Impacto social"]),
    ("¿Con qué palabra te identificas más?", ["Lógico", "Creativo",
    "Práctico", "Empático"])
```

En esta parte se define el cuestionario a usar: `preguntas` es una lista de tuplas en la cual cada una tiene la pregunta (string) y una lista de respuestas posibles. Se presentan 6 preguntas, cada una diseñada para asociarse a uno de los 4 perfiles vocacionales: científico, artístico, técnico o social.

Paso 3. Mapa de correspondencias entre respuestas y perfiles

```

perfil_map = {
    "Leer artículos científicos": "científico",
    "Matemáticas": "científico",
    "Analizando datos": "científico",
    "Resolver acertijos": "científico",
    "Descubrimiento": "científico",
    "Lógico": "científico",

    "Pintar o dibujar": "artístico",
    "Arte": "artístico",
    "Creando cosas nuevas": "artístico",
    "Tocar un instrumento": "artístico",
    "Expresión": "artístico",
    "Creativo": "artístico",

    "Reparar cosas": "técnico",
    "Tecnología": "técnico",
    "Con herramientas": "técnico",
    "Armar circuitos": "técnico",
    "Precisión": "técnico",
    "Práctico": "técnico",

    "Ayudar a personas": "social",
    "Psicología": "social",
    "En equipo con personas": "social",
    "Voluntariado": "social",
    "Impacto social": "social",
    "Empático": "social"
}

```

En esta parte se define el mapeo de correspondencias entre las respuestas y los perfiles.

- `perfil_map` es un diccionario que asocia cada respuesta con un perfil vocacional.
- Cuando un estudiante selecciona una opción, se usa esta estructura para saber a qué perfil suma puntos.

Paso 4. Recomendaciones según perfil

```

recomendaciones = {
    "científico": "☐ Perfil Científico: Podrías destacar en áreas como Física, Matemáticas, Biología, Ingeniería o Investigación.",
    "artístico": "☐ Perfil Artístico: Podrías sobresalir en Diseño, Música, Artes Visuales, Publicidad o Cine.",
}

```

```

    "técnico": "□ Perfil Técnico: Carreras como Mecatrónica, Robótica,
Sistemas o Mantenimiento son una excelente opción.",
    "social": "□ Perfil Social: Psicología, Educación, Trabajo Social o
Comunicación podrían ser tu vocación."
}

```

En esta parte se definen las recomendaciones según los perfiles. Este diccionario contiene la retroalimentación final que se mostrará al usuario según su perfil dominante. Las recomendaciones están escritas en un lenguaje claro y orientado a la acción (carreras sugeridas).

Paso 5. Variables de control

```

indice_pregunta = 0
respuestas = []
conteo = {"científico": 0, "artístico": 0, "técnico": 0, "social": 0}

```

En esta parte se definen las variables de control:

- `indice_pregunta`: controla qué pregunta está activa actualmente.
- `respuestas`: lista de respuestas seleccionadas.
- `conteo`: diccionario que cuenta cuántas veces cada perfil ha sido seleccionado.

Paso 6. Creación de widgets (interfaz)

```

label_pregunta = widgets.Label()
radio_respuesta = widgets.RadioButtons()
boton_siguiente = widgets.Button(description="Siguiente")
barra = widgets.IntProgress(min=0, max=len(preguntas), value=0)
salida = widgets.Output()

```

En esta parte se crea la interfaz con la siguiente estructura:

- `label_pregunta`: muestra la pregunta actual como texto.
- `radio_respuesta`: widget para que el estudiante elija una opción.
- `boton_siguiente`: botón para pasar a la siguiente pregunta.
- `barra`: barra de progreso que avanza a medida que se responde el test.
- `salida`: contenedor visual donde se mostrará el resultado final.

Paso 7. Mostrar la pregunta actual

```

def mostrar_pregunta():
    label_pregunta.value = preguntas[indice_pregunta][0]
    radio_respuesta.options = preguntas[indice_pregunta][1]
    barra.value = indice_pregunta

```

En esta parte se define la función que representa la pregunta actual:

- Cambia el texto del `label_pregunta` con la pregunta correspondiente.
- Actualiza las opciones del `radio_respuesta`.
- Avanza visualmente la barra de progreso.

Paso 8. Función para avanzar a la siguiente pregunta

```
def siguiente(_):
    global indice_pregunta
    seleccion = radio_respuesta.value
    respuestas.append(seleccion)
    perfil = perfil_map.get(seleccion)
    if perfil:
        conteo[perfil] += 1

    indice_pregunta += 1
    if indice_pregunta < len(preguntas):
        mostrar_pregunta()
    else:
        mostrar_resultado()
```

En esta parte se define la función para avanzar a la siguiente pregunta, con las siguientes características:

- Obtiene la respuesta elegida.
- La guarda en la lista `respuestas`.
- Usa `perfil_map` para determinar a qué perfil pertenece.
- Suma 1 punto al perfil correspondiente.
- Avanza a la siguiente pregunta o, si ya no hay más, muestra el resultado.

Paso 9. Mostrar el resultado final

```
def mostrar_resultado():
    salida.clear_output()
    with salida:
        print("✓ Test completado.\n")
        perfil_final = max(conteo, key=conteo.get)
        print(f"□ Tu perfil vocacional dominante es:
**{perfil_final.upper()}**\n")
        print(recomendaciones[perfil_final])
        print("\nRespuestas seleccionadas:")
        for i, r in enumerate(respuestas):
            print(f"{i+1}. {preguntas[i][0]} → {r}")
        boton_siguiente.disabled = True
```

En esta parte se define la función que muestra el resultado final, con las siguientes características:

- Limpia la salida anterior.
- Encuentra el perfil con mayor puntuación (`max(conteo, key=conteo.get)`).

- Muestra el perfil dominante y su recomendación.
- Lista todas las respuestas dadas.
- Desactiva el botón de "Siguiente" para evitar más clics tras terminar.

Paso 10. Botón para reiniciar el test

```
def reiniciar(_):
    global indice_pregunta, respuestas, conteo
    indice_pregunta = 0
    respuestas = []
    conteo = {k: 0 for k in conteo}
    boton_siguiente.disabled = False
    salida.clear_output()
    mostrar_pregunta()
```

En esta parte se define la función que permitirá reiniciar el test. Reinicia todas las variables al estado inicial. Limpia la salida y vuelve a mostrar la primera pregunta.

```
boton_reiniciar = widgets.Button(description="🔄 Reiniciar")
boton_reiniciar.on_click(reiniciar)
```

Esta parte crea el botón "Reiniciar" y conecta su acción al evento `reiniciar`.

Paso 11. Conectar evento del botón "Siguiente"

```
boton_siguiente.on_click(siguiente)
```

En esta parte, cuando el botón "Siguiente" es presionado, se ejecuta la función `siguiente`.

Paso 12. Mostrar toda la interfaz

```
mostrar_pregunta()
display(barra, label_pregunta, radio_respuesta, boton_siguiente, salida,
boton_reiniciar)
```

Esta última parte del código, lo que hace es que muestra la primera pregunta y renderiza todos los elementos de la interfaz en el orden adecuado.

Como conclusión final para los docentes, este tipo de test es ideal para:

- Diagnóstico vocacional rápido en el aula o en línea.
- Actividades de orientación o tutoría en carreras.
- Introducción a lenguajes como Python para crear herramientas educativas simples.
- Ser adaptado para evaluar intereses profesionales, estilos de aprendizaje, u otras dimensiones psicológicas o educativas.

Actividad práctica 24: Chatbot para practicar derivadas

Objetivo: Diseñar un chatbot interactivo que guíe a los estudiantes en la práctica de derivadas de funciones, ofreciendo retroalimentación inmediata y explicaciones paso a paso para fortalecer el aprendizaje autónomo de conceptos del cálculo diferencial.

Alcance: introducción de conceptos básicos de derivación simbólica usando SymPy, implementación de un chatbot educativo simple usando ipywidgets, simular una tutoría automatizada donde se presentan ejercicios uno a uno, el estudiante responde con expresiones simbólicas, el sistema evalúa y responde si es o no correcta la respuesta del usuario y se proporciona una guía paso a paso de la derivación correcta.

Producto final: un notebook funcional que contenga una app interactiva donde los usuarios practican derivadas simbólicas, reciben retroalimentación. Automática (correcto o incorrecto), visualizan los pasos detallados de la solución correcta e interactúan mediante botones y campos de entrada amigables.

Código:

```
import sympy as sp
import ipywidgets as widgets
from IPython.display import display, clear_output
```

```

# Variables simbólicas
x = sp.Symbol('x')

# Lista de ejercicios con funciones y explicaciones paso a paso
ejercicios = [
    {
        'funcion': sp.sin(x),
        'respuesta': sp.diff(sp.sin(x), x),
        'pasos': "Paso 1: Identifica la función como sin(x)\nPaso 2:
Aplica la regla d/dx[sin(x)] = cos(x)"
    },
    {
        'funcion': x**2,
        'respuesta': sp.diff(x**2, x),
        'pasos': "Paso 1: Es una potencia\nPaso 2: Aplica d/dx[x^n] =
n*x^(n-1)\nResultado: 2*x"
    },
    {
        'funcion': sp.exp(x),
        'respuesta': sp.diff(sp.exp(x), x),
        'pasos': "Paso 1: La función es exponencial\nPaso 2: d/dx[e^x] =
e^x"
    },
    {
        'funcion': sp.ln(x),
        'respuesta': sp.diff(sp.ln(x), x),
        'pasos': "Paso 1: La función es logarítmica\nPaso 2: d/dx[ln(x)]
= 1/x"
    },
    {
        'funcion': x * sp.sin(x),
        'respuesta': sp.diff(x * sp.sin(x), x),
        'pasos': "Paso 1: Identifica como un producto u = x, v =
sin(x)\n"
        "Paso 2: Aplica la regla del producto: u'v + uv'\n"
        "Paso 3: Derivada de x = 1, derivada de sin(x) =
cos(x)\n"
        "Resultado: 1·sin(x) + x·cos(x) = sin(x) + x·cos(x)"
    }
]

# Widgets
indice = 0
entrada = widgets.Text(placeholder='Escribe tu derivada aquí',
layout=widgets.Layout(width='100%'))

```

```

boton_enviar = widgets.Button(description='Verificar',
button_style='info')
boton_siguiete = widgets.Button(description='Siguiete ejercicio',
disabled=True, button_style='success')
boton_ver_pasos = widgets.Button(description='Ver solución paso a paso',
button_style='warning')
salida = widgets.Output()

def mostrar_ejercicio():
    salida.clear_output()
    entrada.value = ""
    boton_siguiete.disabled = True
    boton_ver_pasos.disabled = False
    with salida:
        print(f"□ Ejercicio {indice + 1} de {len(ejercicios)}")
        print("Deriva la siguiete función con respecto a x:")
        sp.pprint(ejercicios[indice]['funcion'])

def verificar(b):
    respuesta_usuario = entrada.value
    correcta = ejercicios[indice]['respuesta']
    try:
        derivada_usuario = sp.sympify(respuesta_usuario)
        if sp.simplify(derivada_usuario - correcta) == 0:
            mensaje = "✓ ¡Correcto!"
        else:
            mensaje = "✗ Incorrecto. Intenta de nuevo o haz clic en 'Ver
solución paso a paso'."
    except Exception as e:
        mensaje = f"□ Error en la interpretación: {e}"

    with salida:
        print(mensaje)
    boton_siguiete.disabled = False

def siguiete(b):
    global indice
    if indice < len(ejercicios) - 1:
        indice += 1
        mostrar_ejercicio()
    else:
        with salida:
            print("□ Has completado todos los ejercicios.")
        boton_ver_pasos.disabled = False

```

```

def mostrar_pasos(b):
    pasos = ejercicios[indice]['pasos']
    with salida:
        print(f"□ Pasos para derivar:")
        print(pasos)
    boton_ver_pasos.disabled = True

# Eventos
boton_enviar.on_click(verificar)
boton_siguiete.on_click(siguiete)
boton_ver_pasos.on_click(mostrar_pasos)

# Interfaz
ui = widgets.VBox([
    salida,
    entrada,
    widgets.HBox([boton_enviar, boton_ver_pasos, boton_siguiete])
])

display(ui)
mostrar_ejercicio()

```

Explicación detallada:

Paso 1. Importación de bibliotecas

```

import sympy as sp
import ipywidgets as widgets
from IPython.display import display, clear_output

```

En esta parte se importan todas las bibliotecas necesarias:

- **sympy:** biblioteca de álgebra simbólica, usada para calcular derivadas, simplificar expresiones, etc.
- **ipywidgets:** permite crear una interfaz con botones, campos de entrada, etc.
- **display, clear_output:** funciones para mostrar y actualizar elementos en celdas interactivas.

Paso 2. Declaración de la variable simbólica

```

x = sp.Symbol('x')

```

En esta línea se define la variable simbólica x para usarla en funciones matemáticas. Esto es esencial para que `sympy` reconozca que debe trabajar de manera simbólica (no numérica).

Paso 3. Lista de ejercicios

```
ejercicios = [
    {
        'funcion': sp.sin(x),
        'respuesta': sp.diff(sp.sin(x), x),
        'pasos': "Paso 1: Identifica la función como sin(x)\nPaso 2:
Aplica la regla d/dx[sin(x)] = cos(x)"
    },
    {
        'funcion': x**2,
        'respuesta': sp.diff(x**2, x),
        'pasos': "Paso 1: Es una potencia\nPaso 2: Aplica d/dx[x^n] =
n*x^(n-1)\nResultado: 2*x"
    },
    {
        'funcion': sp.exp(x),
        'respuesta': sp.diff(sp.exp(x), x),
        'pasos': "Paso 1: La función es exponencial\nPaso 2: d/dx[e^x] =
e^x"
    },
    {
        'funcion': sp.ln(x),
        'respuesta': sp.diff(sp.ln(x), x),
        'pasos': "Paso 1: La función es logarítmica\nPaso 2: d/dx[ln(x)]
= 1/x"
    },
    {
        'funcion': x * sp.sin(x),
        'respuesta': sp.diff(x * sp.sin(x), x),
        'pasos': "Paso 1: Identifica como un producto u = x, v =
sin(x)\n"
        "Paso 2: Aplica la regla del producto: u'v + uv'\n"
        "Paso 3: Derivada de x = 1, derivada de sin(x) =
cos(x)\n"
        "Resultado: 1·sin(x) + x·cos(x) = sin(x) + x·cos(x)"
    }
]
```

En esta parte se define cada ejercicio con un diccionario con 3 claves:

- 'funcion': expresión simbólica a derivar (usando `sympy`).
- 'respuesta': derivada exacta calculada por `sp.diff(...)`.
- 'pasos': texto que explica el procedimiento paso a paso, ideal para aprendizaje guiado.
- Se incluyen funciones comunes: seno, potencia, exponencial, logaritmo, y producto de funciones.

Paso 4. Widgets de la interfaz

```

indice = 0
entrada = widgets.Text(placeholder='Escribe tu derivada aquí',
layout=widgets.Layout(width='100%'))
boton_enviar = widgets.Button(description='Verificar',
button_style='info')
boton_siguiete = widgets.Button(description='Siguiente ejercicio',
disabled=True, button_style='success')
boton_ver_pasos = widgets.Button(description='Ver solución paso a paso',
button_style='warning')
salida = widgets.Output()

```

En esta parte se definen los widgets para la interfaz, con los siguientes elementos:

- `indice`: controla qué ejercicio está activo.
- `entrada`: campo de texto donde el usuario escribe la derivada.
- `boton_enviar`: verifica si la derivada escrita es correcta.
- `boton_siguiete`: avanza al siguiente ejercicio.
- `boton_ver_pasos`: muestra los pasos de resolución.
- `salida`: área donde se muestran los mensajes, resultados, pasos, etc.

Paso 5. Función para mostrar el ejercicio

```

def mostrar_ejercicio():
    salida.clear_output()
    entrada.value = ""
    boton_siguiete.disabled = True
    boton_ver_pasos.disabled = False
    with salida:
        print(f"□ Ejercicio {indice + 1} de {len(ejercicios)}")
        print("Deriva la siguiente función con respecto a x:")
        sp.pprint(ejercicios[indice]['funcion'])

```

En esta parte se define la función para mostrar el ejercicio con base a la siguiente estructura:

- Limpia la salida anterior.
- Resetea el campo de entrada.
- Desactiva el botón "Siguiente" (hasta verificar).

- Muestra el número de ejercicio actual.
- Muestra la función a derivar en formato matemático (con `pprint` de SymPy).

Paso 6. Función para verificar la derivada

```
def verificar(b):
    respuesta_usuario = entrada.value
    correcta = ejercicios[indice]['respuesta']
    try:
        derivada_usuario = sp.sympify(respuesta_usuario)
        if sp.simplify(derivada_usuario - correcta) == 0:
            mensaje = "✓ ¡Correcto!"
        else:
            mensaje = "✗ Incorrecto. Intenta de nuevo o haz clic en 'Ver
solución paso a paso'."
    except Exception as e:
        mensaje = f"❏ Error en la interpretación: {e}"

    with salida:
        print(mensaje)
    boton_siguiete.disabled = False
```

En esta parte se define la función que permite verificar la derivada.

1. Toma el texto ingresado por el usuario.
2. Intenta convertirlo a una expresión simbólica con `sp.sympify()`.
3. Compara la derivada del usuario con la derivada correcta:
 - Se resta una de la otra.
 - Si la diferencia se simplifica a 0, es correcta.
4. Si hay error de sintaxis (ej. `x**`), se muestra el mensaje de error.
5. Muestra el resultado en pantalla y habilita el botón "Siguiete".

Paso 7. Función para avanzar al siguiente ejercicio

```
def siguiente(b):
    global indice
    if indice < len(ejercicios) - 1:
        indice += 1
        mostrar_ejercicio()
    else:
        with salida:
            print("❏ Has completado todos los ejercicios.")
        boton_ver_pasos.disabled = False
```

En esta parte se define la función que permite avanzar al siguiente ejercicio:

- Incrementa el índice del ejercicio.
- Muestra el siguiente si hay más.
- Si ya se completaron todos, muestra un mensaje de cierre.
- Reactiva el botón para ver los pasos en el nuevo ejercicio.

Paso 8. Mostrar la solución paso a paso

```
def mostrar_pasos(b):  
    pasos = ejercicios[indice]['pasos']  
    with salida:  
        print(f"□ Pasos para derivar:")  
        print(pasos)  
    boton_ver_pasos.disabled = True
```

Esta función:

- Extrae el texto explicativo del ejercicio actual.
- Lo imprime en pantalla para que el usuario pueda aprender del procedimiento.
- Desactiva el botón para evitar repetir el mismo mensaje.

Paso 9. Conectar botones a funciones

```
boton_enviar.on_click(verificar)  
boton_siguiete.on_click(siguiete)  
boton_ver_pasos.on_click(mostrar_pasos)
```

En esta parte se asocia cada botón a su función correspondiente:

- Verificar → `verificar`
- Siguiente ejercicio → `siguiete`
- Ver solución paso a paso → `mostrar_pasos`

Paso 10. Interfaz visual

```
ui = widgets.VBox([  
    salida,  
    entrada,  
    widgets.HBox([boton_enviar, boton_ver_pasos, boton_siguiete])  
)  
  
display(ui)  
mostrar_ejercicio()
```

Por último, en esta parte:

- Se organiza la interfaz usando cajas verticales (`VBox`) y horizontales (`HBox`).
- Primero se muestra la zona de salida (`salida`), luego el campo de entrada y botones alineados.
- Finalmente, se muestra todo con `display(ui)` y se llama al primer ejercicio con `mostrar_ejercicio()`.

Actividad práctica 25: Planificador inteligente de horarios de clase

Objetivo: Asistir en la planificación de horarios evitando choques entre grupos, docentes y salones.

Alcance: simulación con matrices lógicas, visualización de conflicto de horarios, interfaz para ingresar clase, docente y hora, mensaje de advertencia si hay conflicto de horario.

Producto final: una app interactiva para registrar los datos de una clase y detectar si hay conflictos en tiempo real si hay traslape de horarios en la misma aula o por el mismo docente.

Código:

```
import ipywidgets as widgets
from IPython.display import display, clear_output
import pandas as pd
from datetime import datetime

# Base de datos vacía
horarios_df = pd.DataFrame(columns=["Día", "Hora Inicio", "Hora Fin",
"Docente", "Clase", "Aula"])
```

```

# Opciones posibles
dias = ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes']
horas = ['08:00', '09:00', '10:00', '11:00', '12:00', '13:00', '14:00',
'15:00', '16:00', '17:00']
aulas = ['A101', 'B202', 'C303']

# Widgets de entrada
dia = widgets Dropdown(options=dias, description="Día")
hora_inicio = widgets Dropdown(options=horas, description="Inicio")
hora_fin = widgets Dropdown(options=horas, description="Fin")
docente = widgets Text(description="Docente")
clase = widgets Text(description="Clase")
aula = widgets Dropdown(options=aulas, description="Aula")

# Botones
btn_agregar = widgets.Button(description="Agregar clase")
btn_ver = widgets.Button(description="Ver horario")
salida = widgets.Output()

# Función para verificar traslape de horarios
def hay_conflicto(fila_nueva, horarios):
    nuevo_inicio = datetime.strptime(fila_nueva["Hora Inicio"],
"%H:%M")
    nuevo_fin = datetime.strptime(fila_nueva["Hora Fin"], "%H:%M")
    for _, fila in horarios.iterrows():
        inicio = datetime.strptime(fila["Hora Inicio"], "%H:%M")
        fin = datetime.strptime(fila["Hora Fin"], "%H:%M")

        # Comprobamos traslape
        traslape = (fila_nueva["Día"] == fila["Día"]) and (
            (inicio <= nuevo_inicio < fin) or (inicio < nuevo_fin <=
fin) or
            (nuevo_inicio <= inicio and nuevo_fin >= fin)
        )

        if traslape:
            # Verificamos si es conflicto por aula o por docente
            if fila_nueva["Aula"] == fila["Aula"]:
                return f"✗ Conflicto de aula: el aula {fila['Aula']}
ya está ocupada en ese horario."
            if fila_nueva["Docente"] == fila["Docente"]:
                return f"✗ Conflicto de docente: {fila['Docente']} ya
tiene clase asignada en ese horario."
    return None

```

```

# Función principal para agregar clases
def agregar_clase(_):
    global horarios_df
    salida.clear_output()

    fila_nueva = {
        "Día": dia.value,
        "Hora Inicio": hora_inicio.value,
        "Hora Fin": hora_fin.value,
        "Docente": docente.value,
        "Clase": clase.value,
        "Aula": aula.value
    }

    try:
        hi = datetime.strptime(hora_inicio.value, "%H:%M")
        hf = datetime.strptime(hora_fin.value, "%H:%M")
    except:
        with salida:
            print("❌ Error en formato de hora.")
            return

    if hi >= hf:
        with salida:
            print("❌ La hora de fin debe ser mayor que la hora de inicio.")
            return

    conflicto = hay_conflicto(fila_nueva, horarios_df)

    with salida:
        if conflicto:
            print(conflicto)
        else:
            horarios_df.loc[len(horarios_df)] = fila_nueva
            print("✅ Clase agregada correctamente.")

# Función para mostrar horario
def ver_horario(_):
    salida.clear_output()
    with salida:
        if horarios_df.empty:
            print("❌ No hay clases asignadas.")
        else:

```

```

        display(horarios_df.sort_values(by=["Día", "Hora
Inicio"]))

# Eventos
btn_agregar.on_click(agregar_clase)
btn_ver.on_click(ver_horario)

# Mostrar interfaz
display(dia, hora_inicio, hora_fin, aula, docente, clase, btn_agregar,
btn_ver, salida)

```

Actividad práctica 26: Chatbot recomendador de recursos educativos personalizados

Objetivo: identificar el estilo de aprendizaje predominante del estudiante (visual, auditivo, kinestésico, mixto) y recomendar recursos educativos personalizados.

Alcance: uso de interfaz gráfica con botones, simulación de un test breve de diagnóstico, clasificación del estilo de aprendizaje de acuerdo a las respuestas del usuario y recomendación automática de recursos útiles (videos, podcast, actividades prácticas, etc).

Producto final: un chatbot interactivo para estudiantes que respondan preguntas tipo test y reciben sugerencias de estudio adaptadas a su estilo de aprendizaje.

Código:

```

import ipywidgets as widgets
from IPython.display import display, clear_output

# Preguntas y opciones
preguntas = [
    {
        "pregunta": "¿Qué método prefieres para aprender un nuevo tema?",

```

```

        "opciones": {
            "Ver un video o diagrama": "visual",
            "Escuchar una explicación o podcast": "auditivo",
            "Hacer una maqueta o actividad física": "kinestesico"
        }
    },
    {
        "pregunta": "Cuando estudias, ¿qué haces más a menudo?",
        "opciones": {
            "Subrayo y hago mapas mentales": "visual",
            "Leo en voz alta o escucho audios": "auditivo",
            "Hago ejercicios, esquemas o uso las manos": "kinestesico"
        }
    },
    {
        "pregunta": "¿Cómo recuerdas mejor la información?",
        "opciones": {
            "Viendo imágenes o gráficas": "visual",
            "Escuchando la información": "auditivo",
            "Repitiendo o haciendo cosas con ella": "kinestesico"
        }
    },
    {
        "pregunta": "¿Qué actividad disfrutas más al aprender?",
        "opciones": {
            "Ver películas o leer cómics": "visual",
            "Escuchar historias o canciones": "auditivo",
            "Experimentar o participar en juegos": "kinestesico"
        }
    }
]

```

```

# Contadores

```

```

puntajes = {"visual": 0, "auditivo": 0, "kinestesico": 0}
indice_actual = 0

```

```

salida = widgets.Output()

```

```

def mostrar_pregunta(indice):
    salida.clear_output()
    with salida:
        pregunta = preguntas[indice]
        print(f"Pregunta {indice+1} de
{len(preguntas)}:\n{pregunta['pregunta']}")
        botones = []

```

```

        for texto, estilo in pregunta["opciones"].items():
            btn = widgets.Button(description=texto,
                                layout=widgets.Layout(width='auto'))
            def on_click(b, estilo=estilo):
                puntajes[estilo] += 1
                siguiente_pregunta()
            btn.on_click(on_click)
            botones.append(btn)
        display(widgets.VBox(botones))

def siguiente_pregunta():
    global indice_actual
    indice_actual += 1
    if indice_actual < len(preguntas):
        mostrar_pregunta(indice_actual)
    else:
        mostrar_resultado()

def mostrar_resultado():
    salida.clear_output()
    with salida:
        max_estilo = max(puntajes, key=puntajes.get)
        total = sum(puntajes.values())
        porcentaje = {k: int(100*v/total) for k, v in puntajes.items()}
        print("✓ Diagnóstico completado. Resultado:")
        print("- Visual:", porcentaje["visual"], "%")
        print("- Auditivo:", porcentaje["auditivo"], "%")
        print("- Kinestésico:", porcentaje["kinestesico"], "%\n")

        if list(puntajes.values()).count(puntajes[max_estilo]) > 1:
            estilo = "mixto"
            print("□ Tu estilo de aprendizaje es **Mixto**.")
        else:
            estilo = max_estilo
            print(f"□ Tu estilo predominante es
**{estilo.capitalize()}**.\n")

        print("□ Recomendaciones educativas según tu estilo:\n")

        if estilo == "visual":
            print("- Usa mapas mentales, diagramas y videos
explicativos.")
            print("- Prueba con plataformas como Khan Academy o YouTube
Educativo.")
        elif estilo == "auditivo":

```

```

        print("- Escucha podcasts educativos y graba tus notas.")
        print("- Usa apps como Audible o podcasts de Spotify.")
    elif estilo == "kinestesico":
        print("- Realiza actividades prácticas, juegos,
simulaciones.")
        print("- Usa kits de experimentación o Arduino para aprender
tocando.")
    else: # Mixto
        print("- Combina videos, audios y actividades prácticas.")
        print("- Usa plataformas con variedad de formatos como
Coursera o Edpuzzle.")

    print("\nGracias por usar el Asesor de Estilos de Aprendizaje ☐")

# Iniciar el test
mostrar_pregunta(indice_actual)
display(salida)

```

Explicación detallada:

Paso 1. Importación de bibliotecas

```

import ipywidgets as widgets
from IPython.display import display, clear_output

```

En esta parte se importan todas las bibliotecas necesarias:

- **ipywidgets:** biblioteca que permite crear interfaces gráficas simples (botones, texto, etc.) dentro de Jupyter o Google Colab.
- **display:** función para mostrar los elementos en la interfaz.
- **clear_output:** limpia la salida actual (para evitar mostrar resultados acumulados).

Paso 2. Definición de preguntas

```

preguntas = [
    {
        "pregunta": "¿Qué método prefieres para aprender un nuevo tema?",
        "opciones": {
            "Ver un video o diagrama": "visual",

```



```

        "Escuchar una explicación o podcast": "auditivo",
        "Hacer una maqueta o actividad física": "kinestesico"
    }
},
{
    "pregunta": "Cuando estudias, ¿qué haces más a menudo?",
    "opciones": {
        "Subrayo y hago mapas mentales": "visual",
        "Leo en voz alta o escucho audios": "auditivo",
        "Hago ejercicios, esquemas o uso las manos": "kinestesico"
    }
},
{
    "pregunta": "¿Cómo recuerdas mejor la información?",
    "opciones": {
        "Viendo imágenes o gráficas": "visual",
        "Escuchando la información": "auditivo",
        "Repitiendo o haciendo cosas con ella": "kinestesico"
    }
},
{
    "pregunta": "¿Qué actividad disfrutas más al aprender?",
    "opciones": {
        "Ver películas o leer cómics": "visual",
        "Escuchar historias o canciones": "auditivo",
        "Experimentar o participar en juegos": "kinestesico"
    }
}
]

```

En esta parte se define el cuestionario. Se define una lista con 4 preguntas. Cada pregunta es un diccionario con:

- "pregunta": el texto que se mostrará al usuario.
- "opciones": un subdiccionario con:
 - Clave: texto que aparecerá en el botón.
 - Valor: tipo de estilo de aprendizaje relacionado.

Paso 3. Contadores y estado inicial

```

puntajes = {"visual": 0, "auditivo": 0, "kinestesico": 0}
indice_actual = 0
salida = widgets.Output()

```

En esta parte se definen los contadores:

- **puntajes:** diccionario que acumula cuántas veces el usuario elige opciones visuales, auditivas o kinestésicas.
- **indice_actual:** controla cuál pregunta se está mostrando actualmente.
- **salida:** widget donde se imprime el contenido (preguntas y resultados).

Paso 4. Mostrar una pregunta

```
def mostrar_pregunta(indice):
    salida.clear_output()
    with salida:
        pregunta = preguntas[indice]
        print(f"Pregunta {indice+1} de
{len(preguntas)}:\n{pregunta['pregunta']}")
        botones = []
```

En esta parte:

- `salida.clear_output()`: limpia el contenido previo.
- Se accede a la pregunta actual.
- Se imprime el número de la pregunta.
- Se crea una lista vacía `botones` que almacenará los botones de respuesta.

Paso 5. Crear botones de respuesta

```
for texto, estilo in pregunta["opciones"].items():
    btn = widgets.Button(description=texto,
layout=widgets.Layout(width='auto'))
```

En esta parte:

- Se itera sobre las opciones.
- `texto`: es lo que se mostrará en el botón.
- `estilo`: es el valor asociado (visual, auditivo, kinestésico).
- Se crea un botón con ese texto.

Paso 6. Manejador de eventos para cada botón

```
def on_click(b, estilo=estilo):
    puntajes[estilo] += 1
    siguiente_pregunta()
    btn.on_click(on_click)
    botones.append(btn)
```

En esta parte:

- Cada botón tiene una función `on_click()` que:
 1. Incrementa el contador del estilo correspondiente.
 2. Llama a `siguiente_pregunta()` para mostrar la siguiente pregunta.

- Luego se conecta el evento (`.on_click(...)`) y se agrega a la lista de botones.

Paso 7. Mostrar todos los botones

```
display(widgets.VBox(botones))
```

Esta línea hace que se muestran los botones en vertical (`VBox`).

Paso 8. Ir a la siguiente pregunta

```
def siguiente_pregunta():
    global indice_actual
    indice_actual += 1
    if indice_actual < len(preguntas):
        mostrar_pregunta(indice_actual)
    else:
        mostrar_resultado()
```

En esta parte se define la función de la siguiente pregunta con base a las siguientes características:

- Se incrementa el índice para pasar a la siguiente pregunta.
- Si todavía hay preguntas, se muestra la siguiente.
- Si se terminan, se muestra el resultado.

Paso 9. Mostrar resultado final

```
def mostrar_resultado():
    salida.clear_output()
    with salida:
        max_estilo = max(puntajes, key=puntajes.get)
        total = sum(puntajes.values())
        porcentaje = {k: int(100*v/total) for k, v in puntajes.items() }
```

En esta parte se define la función que permite mostrar los resultados con las siguientes características:

- Se limpia la salida.
- Se identifica cuál estilo tiene mayor puntaje (`max_estilo`).
- Se calcula el total de respuestas dadas.
- Se calcula el porcentaje para cada estilo.

Paso 10. Imprimir resultado numérico y textual

```
print("✓ Diagnóstico completado. Resultado:")
print("- Visual:", porcentaje["visual"], "%")
print("- Auditivo:", porcentaje["auditivo"], "%")
```

```
print("- Kinestésico:", porcentaje["kinestesico"], "%\n")
```

En esta parte, se muestran los porcentajes para cada estilo.

Paso 11. Determinar si hay empate

```
if list(puntajes.values()).count(puntajes[max_estilo]) > 1:
    estilo = "mixto"
    print("□ Tu estilo de aprendizaje es **Mixto**.")
else:
    estilo = max_estilo
    print(f"□ Tu estilo predominante es
**{estilo.capitalize()}**.\n")
```

En esta parte se considera lo siguiente:

- Si hay empate entre estilos, se considera "mixto".
- Si no, se muestra el estilo predominante.

Paso 12. Mostrar recomendaciones según el estilo

```
print("□ Recomendaciones educativas según tu estilo:\n")

if estilo == "visual":
    print("- Usa mapas mentales, diagramas y videos
explicativos.")
    print("- Prueba con plataformas como Khan Academy o YouTube
Educativo.")
elif estilo == "auditivo":
    print("- Escucha podcasts educativos y graba tus notas.")
    print("- Usa apps como Audible o podcasts de Spotify.")
elif estilo == "kinestesico":
    print("- Realiza actividades prácticas, juegos,
simulaciones.")
    print("- Usa kits de experimentación o Arduino para aprender
tocando.")
else: # Mixto
    print("- Combina videos, audios y actividades prácticas.")
    print("- Usa plataformas con variedad de formatos como
Coursera o Edpuzzle.")
```

En esta parte se muestran las recomendaciones de acuerdo a su estilo; es decir, según el resultado, se dan sugerencias didácticas adaptadas al estilo de aprendizaje del usuario.

Paso 13. Mensaje final

```
print("\nGracias por usar el Asesor de Estilos de Aprendizaje □")
```

Esta línea lo que hace es dar un mensaje final. Se cierra con un agradecimiento, generando una experiencia positiva.

Paso 14. Iniciar el test

```
mostrar_pregunta(indice_actual)
display(salida)
```

Por último, en esta parte se llama a la función para mostrar la primera pregunta y se muestra el área de `salida` donde irán apareciendo preguntas y resultados.

Actividad práctica 27: Chatbot sobre estado emocional y psicológico

Objetivo: Diseñar un chatbot interactivo que permita a los estudiantes reflexionar sobre su estado emocional y psicológico en el contexto académico, brindando retroalimentación automática y servicios de apoyo.

Alcance: cuestionario breve con preguntas tipo test, evaluación de posibles señales de estrés, ansiedad, desmotivación o fatiga académica.

Recomendaciones personalizadas de acuerdo a los resultados.

Producto final: un chatbot con interfaz gráfica en Google Colab que simula una entrevista de bienestar psicológico básico, útil para evaluación temprana del estado emocional de un estudiante.

Código:

```
import ipywidgets as widgets
from IPython.display import display, clear_output
import matplotlib.pyplot as plt
import datetime

# --- Preguntas específicas para estudiantes de ingeniería ---
preguntas = [
    {
```

```

    "pregunta": "¿Sientes que los proyectos o tareas de ingeniería te
sobrepasan?",
    "opciones": {
        "No, los gestiono bien": 0,
        "A veces me estresan": 1,
        "Frecuentemente me agotan": 2,
        "Casi siempre colapsan mis tiempos": 3
    }
},
{
    "pregunta": "¿Te cuesta mantener la concentración al programar o
resolver problemas?",
    "opciones": {
        "Para nada": 0,
        "Solo cuando estoy cansado": 1,
        "Sí, muy seguido": 2,
        "Siempre me distraigo": 3
    }
},
{
    "pregunta": "¿Has sentido frustración al no entender temas de
matemáticas o física?",
    "opciones": {
        "No, los entiendo bien": 0,
        "A veces me frustro": 1,
        "Frecuentemente me frustro": 2,
        "Estoy frustrado constantemente": 3
    }
},
{
    "pregunta": "¿Duermes menos de 6 horas por preparar tareas o
estudiar?",
    "opciones": {
        "No, duermo bien": 0,
        "Ocasionalmente": 1,
        "Casi todos los días": 2,
        "Siempre sacrifico el sueño": 3
    }
},
{
    "pregunta": "¿Sientes que estudiar ingeniería te está agotando
emocionalmente?",
    "opciones": {
        "No, me apasiona": 0,
        "Un poco": 1,

```

```

        "Sí, bastante": 2,
        "Estoy al límite": 3
    }
}

]

# Variables
puntuacion_total = 0
puntuaciones_pregunta = []
indice_pregunta = 0
salida = widgets.Output()

# Mostrar pregunta actual
def mostrar_pregunta(indice):
    salida.clear_output()
    with salida:
        print(f"Pregunta {indice + 1} de
{len(preguntas)}:\n{preguntas[indice]['pregunta']}")
        botones = []
        for texto, valor in preguntas[indice]["opciones"].items():
            btn = widgets.Button(description=texto,
layout=widgets.Layout(width='auto'))
            def on_click(b, val=valor):
                procesar_respuesta(val)
            btn.on_click(on_click)
            botones.append(btn)
        display(widgets.VBox(botones))

# Procesar respuesta y avanzar
def procesar_respuesta(valor):
    global puntuacion_total, indice_pregunta
    puntuacion_total += valor
    puntuaciones_pregunta.append(valor)
    indice_pregunta += 1
    if indice_pregunta < len(preguntas):
        mostrar_pregunta(indice_pregunta)
    else:
        mostrar_resultado()

# Mostrar gráfico
def mostrar_grafico():
    etiquetas = [f"P{i+1}" for i in range(len(preguntas))]
    plt.figure(figsize=(8,4))
    plt.bar(etiquetas, puntuaciones_pregunta, color='steelblue')
    plt.title("Evaluación emocional por pregunta")

```

```

plt.xlabel("Preguntas")
plt.ylabel("Nivel de afectación")
plt.ylim(0, 3.5)
plt.grid(axis='y')
plt.show()

# Mostrar resultado final
def mostrar_resultado():
    salida.clear_output()
    with salida:
        print("☐ Evaluación emocional completada.\n")
        print(f"☐ Puntaje total: {puntuacion_total} /
{len(preguntas)*3}\n")

        if puntuacion_total <= 4:
            estado = "✓ Estado emocional estable."
            recomendaciones = [
                "Sigue con tu ritmo actual, pero no descuides tu
bienestar.",
                "Sigue usando técnicas de organización y estudio
eficiente."
            ]
        elif 5 <= puntuacion_total <= 8:
            estado = "☐ Signos leves de agotamiento académico."
            recomendaciones = [
                "Intenta dividir tus proyectos en tareas pequeñas.",
                "Haz pausas activas y no olvides descansar."
            ]
        elif 9 <= puntuacion_total <= 12:
            estado = "☐ Estado de estrés académico moderado."
            recomendaciones = [
                "Apóyate en tus profesores o tutores para organizar tus
materias.",
                "Practica técnicas de respiración y usa planificadores
semanales.",
                "Evita estudiar de madrugada."
            ]
        else:
            estado = "☐ Riesgo alto de agotamiento o ansiedad académica."
            recomendaciones = [
                "Contacta con un orientador o profesional de salud
mental.",
                "Considera reducir temporalmente tu carga académica.",
                "Recuerda que pedir ayuda también es parte del proceso de
ingeniería."
            ]

```



```

    ]

    print(estado)
    print("\n☐ Recomendaciones:")
    for rec in recomendaciones:
        print("-", rec)

    # Recursos sugeridos
    print("\n☐ Recursos sugeridos para estudiantes de ingeniería:")
    print("- Canal de YouTube: 'Unicoos' (física/matemáticas)")
    print("- Técnica Pomodoro (25 min estudio + 5 descanso)")
    print("- App gratuita: Focus To-Do, Notion, Forest")
    print("- Consulta con tu tutor académico o centro de bienestar universitario\n")

    mostrar_grafico()

    # Exportar resumen sin error
    fecha = datetime.datetime.now().strftime('%Y-%m-%d %H:%M')
    resumen_recomendaciones = "\n- " + "\n- ".join(recomendaciones)
    resumen = (
        f"Evaluación psicológica - Estudiantes de Ingeniería\n"
        f"Fecha: {fecha}\n"
        f"Puntaje total: {puntuacion_total} / {len(preguntas)*3}\n"
        f"Estado: {estado}\n"
        f"Recomendaciones:{resumen_recomendaciones}"
    )

    print("☐ Resumen:")
    print(resumen)

# Iniciar
mostrar_pregunta(indice_pregunta)
display(salida)

```

Explicación detallada:

Paso 1. Importación de bibliotecas

```

import ipywidgets as widgets
from IPython.display import display, clear_output
import matplotlib.pyplot as plt
import datetime

```

En esta parte se deben de importar todas las bibliotecas necesarias:

- `ipywidgets`: permite crear interfaces gráficas interactivas (botones, menús, etc.) dentro de Jupyter/Colab.
- `display` y `clear_output`: son usados para mostrar y actualizar contenidos dentro del notebook.
- `matplotlib.pyplot`: se usa para crear gráficos de barras.
- `datetime`: se usa para registrar fecha y hora al final del test.

Paso 2. Definición del cuestionario

```
preguntas = [ ... ]
```

En esta parte del código se define el cuestionario el cual contiene "pregunta" el cual representa el texto que se mostrará en la pregunta y "opciones" que son las opciones de respuesta con un valor numérico asociado (0 a 3) que indica el grado de afectación emocional.

Paso 3. Variables de control

```
puntuacion_total = 0
puntuaciones_pregunta = []
indice_pregunta = 0
salida = widgets.Output()
```

En esta parte se definen las variables de control del modelo: `puntuacion_total` la cual representa la suma acumulada de las respuestas, `puntuaciones_pregunta` la cual representa la lista para registrar respuestas individuales, `indice_pregunta` el cual controla qué pregunta se está mostrando y `salida` que es la zona de interfaz para mostrar el contenido interactivo.

Paso 4. Función para mostrar preguntas

```
def mostrar_pregunta(indice):
    salida.clear_output()
    with salida:
        print(f"Pregunta {indice + 1} de
        {len(preguntas)}:\n{preguntas[indice]['pregunta']}")
        botones = []
        for texto, valor in preguntas[indice]["opciones"].items():
            btn = widgets.Button(description=texto,
            layout=widgets.Layout(width='auto'))
            def on_click(b, val=valor):
                procesar_respuesta(val)
            btn.on_click(on_click)
            botones.append(btn)
        display(widgets.VBox(botones))
```

En esta parte del código se define la función para mostrar las preguntas con base a la siguiente estructura: primero limpia la pantalla (`clear_output`), segundo muestra la pregunta correspondiente según el índice, tercero genera botones para cada opción de respuesta y por último se usa `btn.on_click(on_click)` para asociar cada botón a una acción. Note que el uso de `val=valor` dentro de `on_click(b, val=valor)` es una técnica para capturar correctamente el valor seleccionado en cada botón.

Paso 5. Procesar la respuesta

```
def procesar_respuesta(valor):
    global puntuacion_total, indice_pregunta
    puntuacion_total += valor
    puntuaciones_pregunta.append(valor)
    indice_pregunta += 1
    if indice_pregunta < len(preguntas):
        mostrar_pregunta(indice_pregunta)
    else:
        mostrar_resultado()
```

En esta parte, se suma la puntuación correspondiente. Se guarda la puntuación en la lista y por último, si quedan preguntas, avanza; si no, llama a `mostrar_resultado()`.

Paso 6. Gráfico de barras

```
def mostrar_grafico():
    etiquetas = [f"P{i+1}" for i in range(len(preguntas))]
    plt.figure(figsize=(8,4))
    plt.bar(etiquetas, puntuaciones_pregunta, color='steelblue')
    plt.title("Evaluación emocional por pregunta")
    plt.xlabel("Preguntas")
    plt.ylabel("Nivel de afectación")
    plt.ylim(0, 3.5)
    plt.grid(axis='y')
    plt.show()
```

En esta parte, se crea un gráfico que muestra el nivel de afectación por pregunta. El eje Y va de 0 a 3.5 para que cada barra refleje las respuestas (0 a 3). Se usa `plt.bar()` con etiquetas como P1, P2, ..., P5.

Paso 7. Resultados y recomendaciones

```
def mostrar_resultado():
    salida.clear_output()
    with salida:
        print("□ Evaluación emocional completada.\n")
```

```

    print(f"□ Puntaje total: {puntuacion_total} /
{len(preguntas)*3}\n")

    if puntuacion_total <= 4:
        estado = "✓ Estado emocional estable."
        recomendaciones = [
            "Sigue con tu ritmo actual, pero no descuides tu
bienestar.",
            "Sigue usando técnicas de organización y estudio
eficiente."
        ]
    elif 5 <= puntuacion_total <= 8:
        estado = "□ Signos leves de agotamiento académico."
        recomendaciones = [
            "Intenta dividir tus proyectos en tareas pequeñas.",
            "Haz pausas activas y no olvides descansar."
        ]
    elif 9 <= puntuacion_total <= 12:
        estado = "□ Estado de estrés académico moderado."
        recomendaciones = [
            "Apóyate en tus profesores o tutores para organizar tus
materias.",
            "Practica técnicas de respiración y usa planificadores
semanales.",
            "Evita estudiar de madrugada."
        ]
    else:
        estado = "□ Riesgo alto de agotamiento o ansiedad académica."
        recomendaciones = [
            "Contacta con un orientador o profesional de salud
mental.",
            "Considera reducir temporalmente tu carga académica.",
            "Recuerda que pedir ayuda también es parte del proceso de
ingeniería."
        ]

    print(estado)
    print("\n□ Recomendaciones:")
    for rec in recomendaciones:
        print("-", rec)

    # Recursos sugeridos
    print("\n□ Recursos sugeridos para estudiantes de ingeniería:")
    print("- Canal de YouTube: 'Unicoos' (física/matemáticas)")
    print("- Técnica Pomodoro (25 min estudio + 5 descanso)")

```

```

print("- App gratuita: Focus To-Do, Notion, Forest")
print("- Consulta con tu tutor académico o centro de bienestar
universitario\n")

mostrar_grafico()

# Exportar resumen sin error
fecha = datetime.datetime.now().strftime('%Y-%m-%d %H:%M')
resumen_recomendaciones = "\n- " + "\n- ".join(recomendaciones)
resumen = (
    f"Evaluación psicológica - Estudiantes de Ingeniería\n"
    f"Fecha: {fecha}\n"
    f"Puntaje total: {puntuacion_total} / {len(preguntas)*3}\n"
    f"Estado: {estado}\n"
    f"Recomendaciones:{resumen_recomendaciones}"
)

print("☐ Resumen:")
print(resumen)

```

En esta parte del código se muestran los resultados finales y las respectivas recomendaciones. Limpia la salida y muestra: puntaje total, diagnóstico emocional basado en rangos de puntaje, recomendaciones personalizadas, recursos útiles, gráfico visual de resultados y un resumen final con fecha y estado emocional. Los rangos son: 0 – 4 (estable), 5 – 8 (signos leves de agotamiento), 9 – 12 (estrés académico moderado), 13 – 15 (riesgo alto de ansiedad o agotamiento).

Paso 8. Inicio del test

```

mostrar_pregunta(indice_pregunta)
display(salida)

```

En esta parte se inicia el chatbot. Se muestra la primera pregunta y se renderiza el área de salida en la celda.

Como conclusión final para los docentes, este tipo de chatbot es una herramienta que:

- Permite monitorear el estado emocional de estudiantes de forma no invasiva.
- Ayuda a detectar signos tempranos de agotamiento.
- Puede integrarse fácilmente con modelos de aprendizaje automático (TensorFlow) para predecir riesgo de abandono en etapas posteriores.

- Sirve como ejemplo de evaluación formativa interactiva y personalizada.